

Immersive Labs - Wireshark



Introduction to Wireshark

1. Familiarise yourself with the Wireshark application and its uses by reading the 'briefing' section.

Wireshark Overview

- **What it is:** A free, open-source packet analyzer (GUI-based). The terminal version is called **TShark**.
- **Primary Use:** Network troubleshooting, protocol development, analysis, and education.

Key Features

- **Capture Sources:** Can capture live traffic from networks (Ethernet, Wi-Fi) or read from saved packet files (PCAP).
- **Filtering:** Uses a powerful display filter language to refine which packets are shown.
- **Protocol Analysis:** Plugins allow for dissecting new protocols. Can detect and even play VoIP calls.
- **Scope:** Can capture raw USB traffic and wireless traffic visible to the monitor.

- **Name Resolution:** Can convert numerical addresses/ports into human-readable names (e.g., 80 → HTTP).

Using the Interface

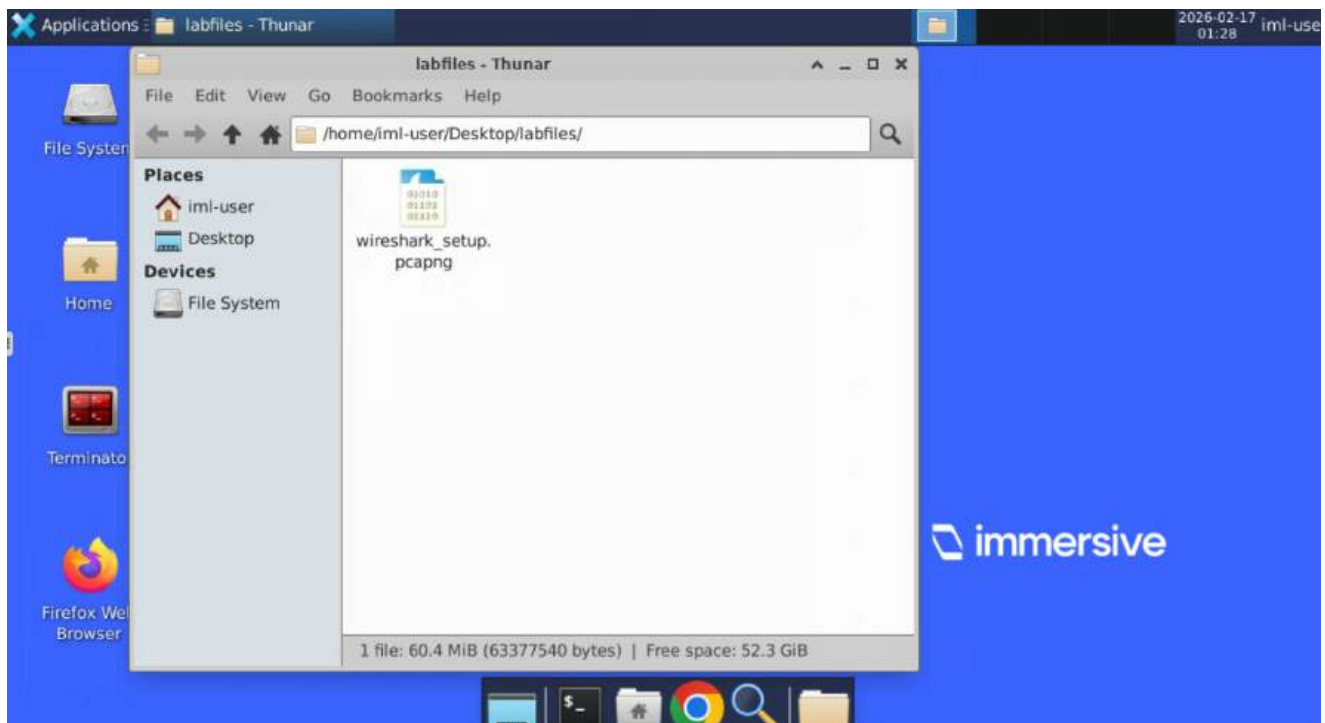
- **Main Panes:** The default view shows a packet list, packet details (tree view), and packet bytes (raw data).
- **Inspection:** Click a packet to expand its details in the tree view to see protocol information.
- **Filter Creation:** Right-click a detail in the packet view and select "**Apply as Filter**" to quickly create filters.

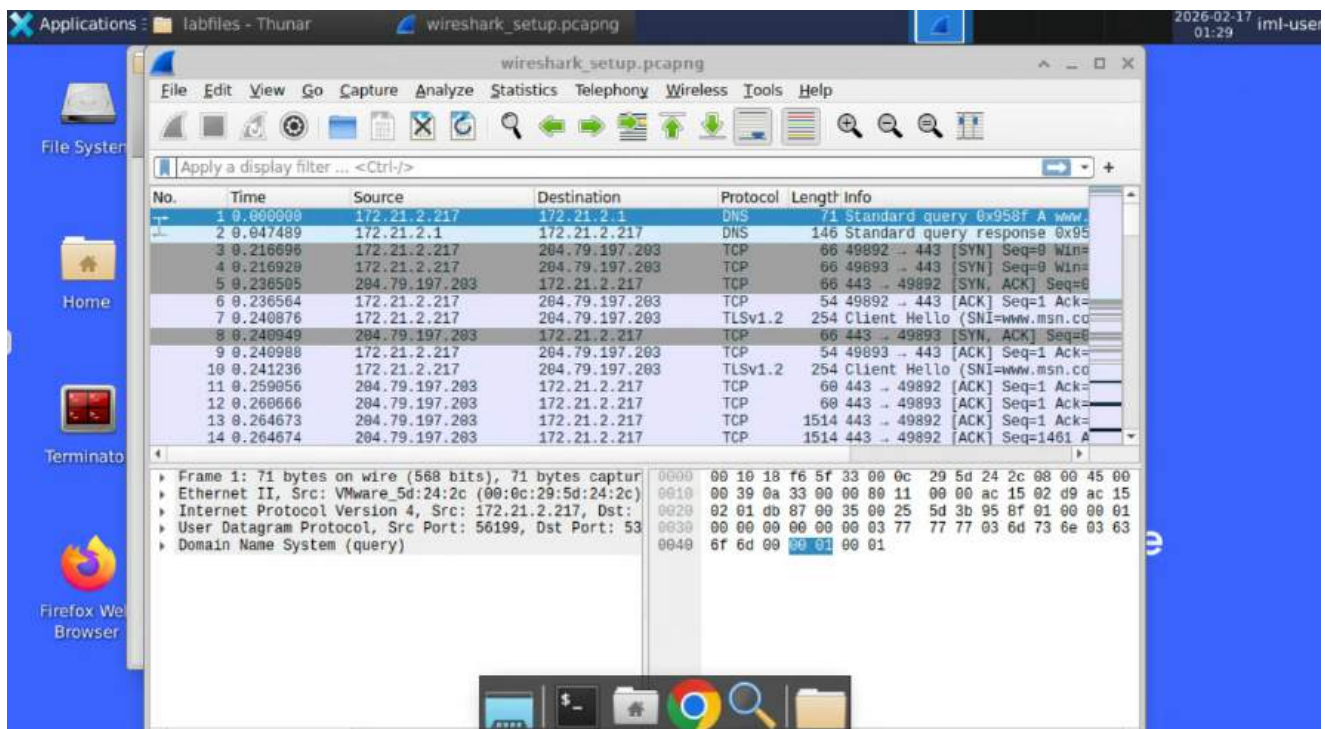
Filtering Basics

- **Purpose:** To search for specific packets (e.g., by protocol, IP, port).
- **Syntax:** Uses fields and operators (e.g., == , && , ||).
Example: `tcp.port eq 161 or icmp` (Shows SNMP or ICMP traffic).
- **Built-in Options:** Can access default filters via **Analyze > Display Filters**.

Investigation Goal: To spot patterns in network traffic and determine if they correspond to legitimate activity or potential abuse.

2. Open the pcap file found in the `labfiles` directory.





3. What is the difference between resolved and unresolved ports on the Wireshark display setup?

- A. Resolved ports display all information about the port (including destination and header data), whereas unresolved sources only show the raw data.
- B. Resolved ports display the name of the well-known service that runs on that port, whereas unresolved ports just display the number.

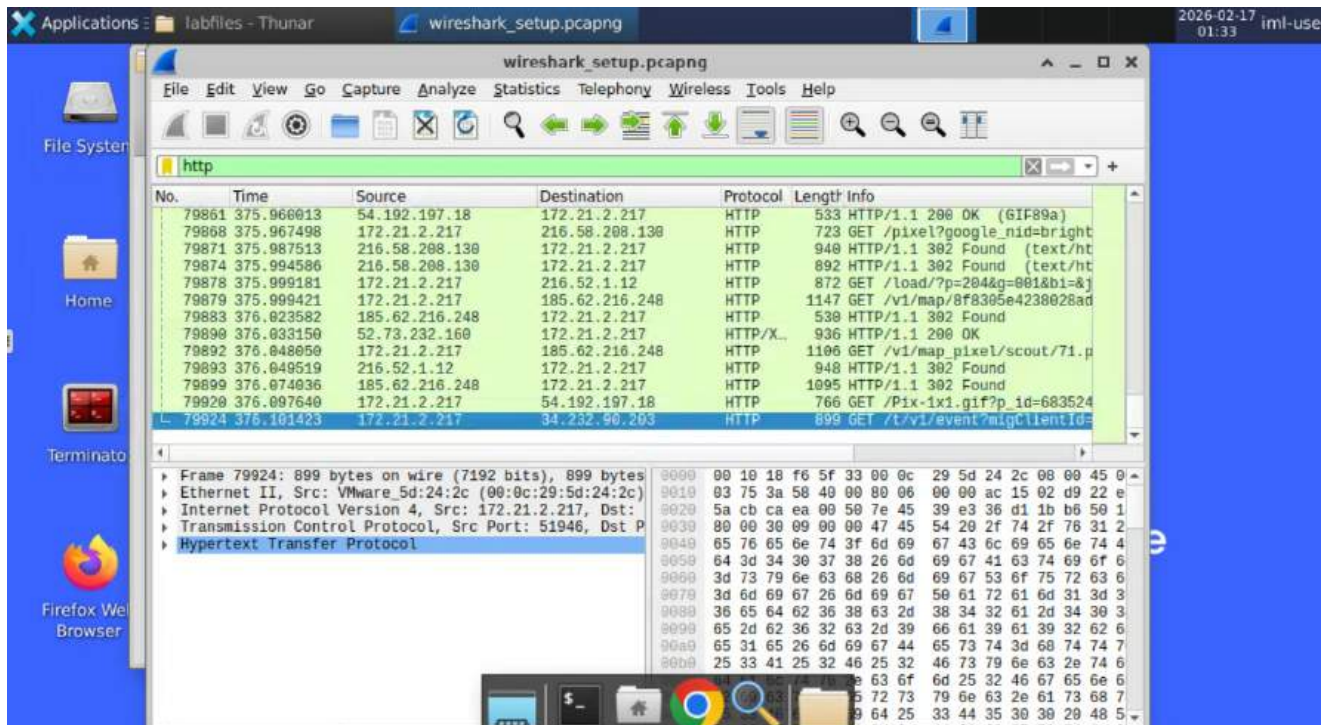
Answer: B

4. What is the correct syntax to use on Wireshark for showing only SMTP and ICMP traffic?

- A. tcpdump.list 25 7
- B. tcp.show smtp & icmp
- C. show 25 & icmp
- D. tcp.port eq 25 or icmp

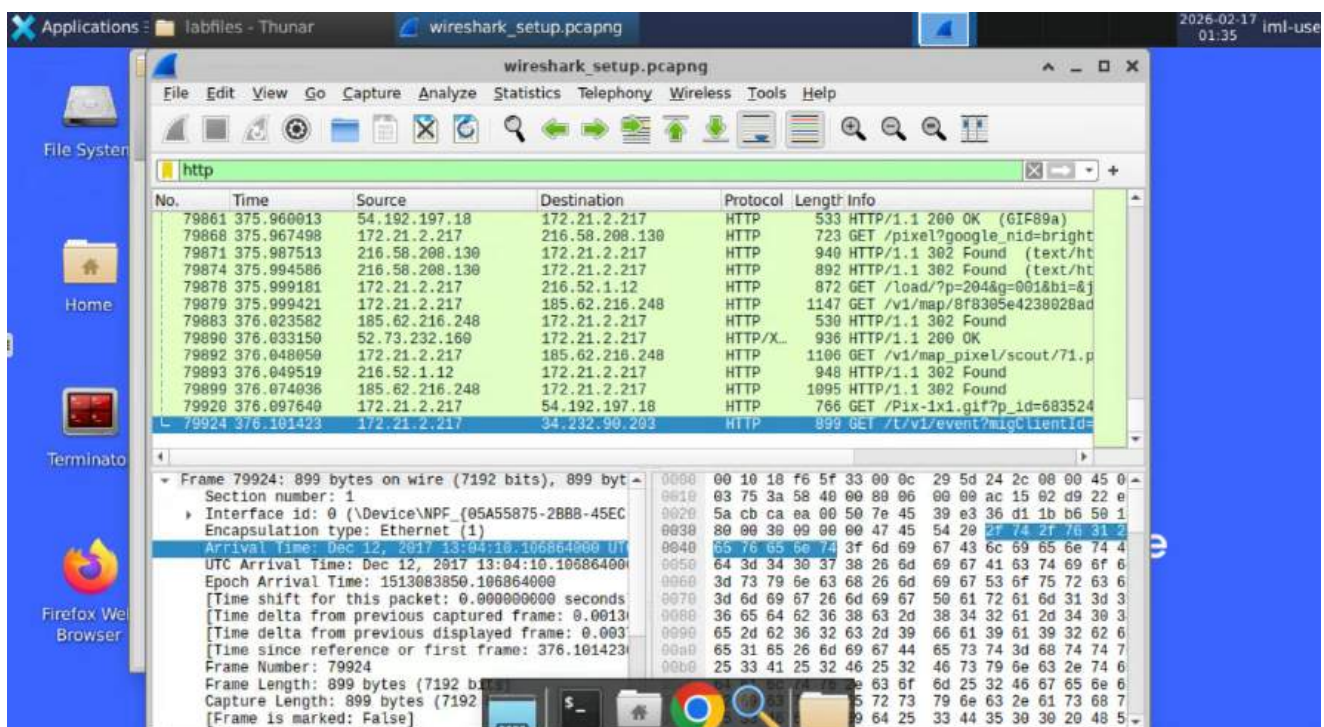
Answer: D

5. Using wireshark_setup.pcapng, filter the packets to view only HTTP requests. What is the source IP address shown on the last packet?



Answer: 172.21.2.217

6. Within that same packet, what is the time shown? Your answer must be in YYYY-MM-DD HH:MM:SS format adjusted for UTC.



Answer: 2017-12-12 13:04:10

7. What is the destination IP address of the last packet?

Answer: 34.232.90.203

Display Filters - Introduction to Filters

1. Understand what Wireshark display filters are and how they are used.

Purpose of Display Filters

- **Why use them:** Network captures (PCAPs) contain大量 noise, even over short periods.
- **Goal:** To filter out irrelevant traffic and focus on data relevant to an investigation, such as **Indicators of Compromise (IOCs)** .
- **Benefit:** Helps identify malicious traffic patterns between systems without manually scrolling through thousands of packets.

Basic Syntax

The fundamental structure for a filter is:

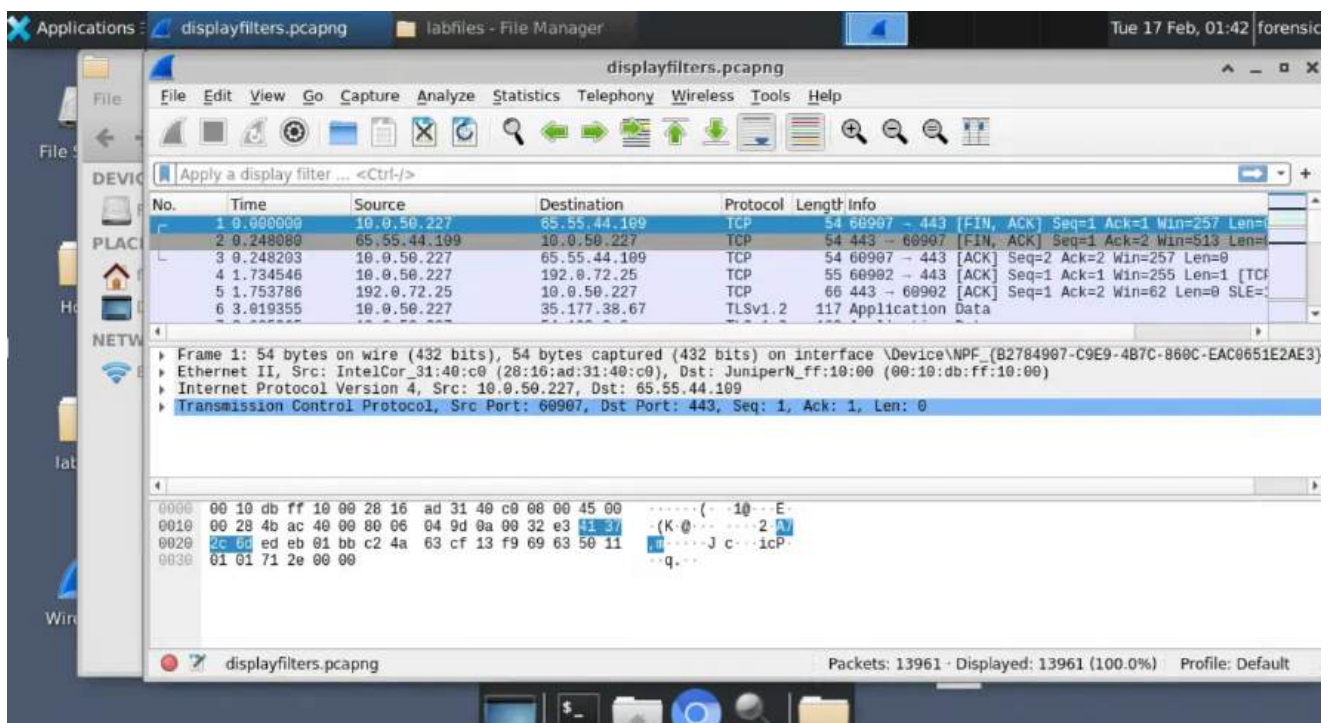
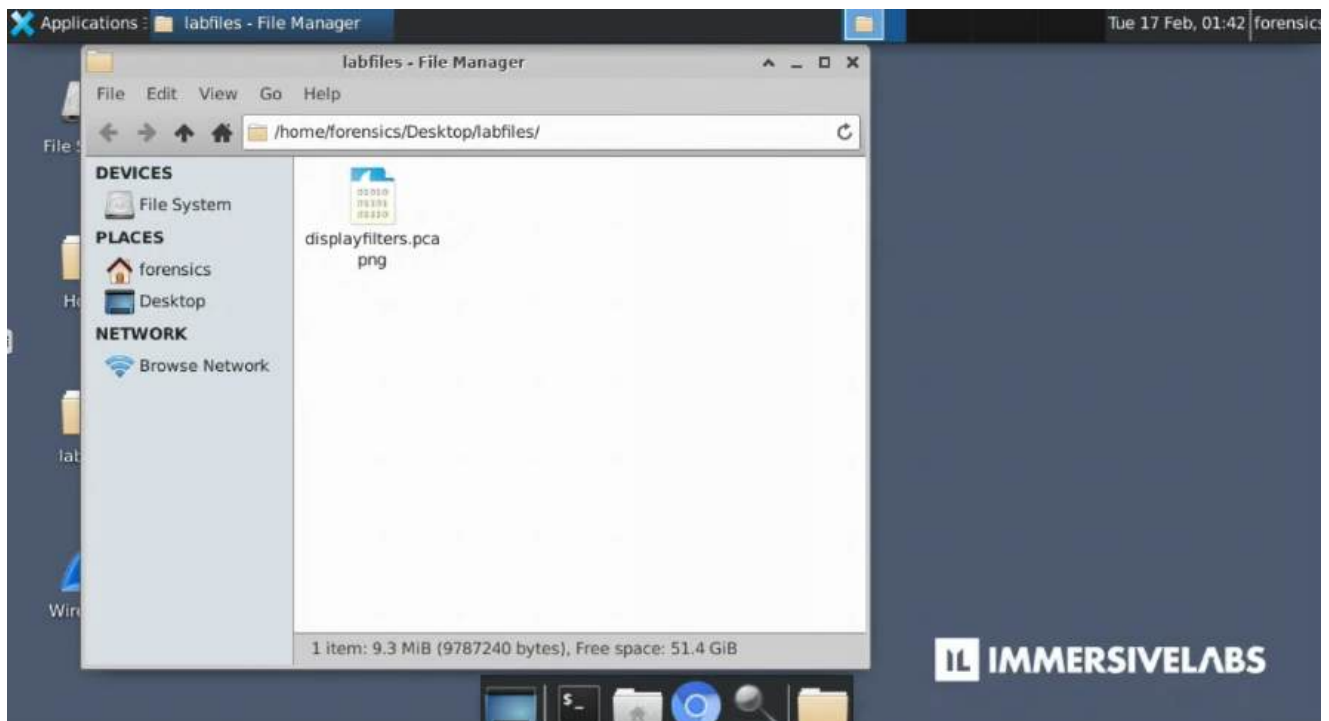
```
<field>.<subfield> <comparison_operator> <value>
```

- **Field:** The protocol or field you want to inspect (e.g., `tcp`, `ip`, `dns`).
- **Comparison Operators:** Used to define the relationship.
 - `eq` or `==` (Equal to)
 - `ne` or `≠` (Not equal to)
- **Logical Operators (for combining):**
 - `and` or `&&`
 - `or` or `||`

Common Examples

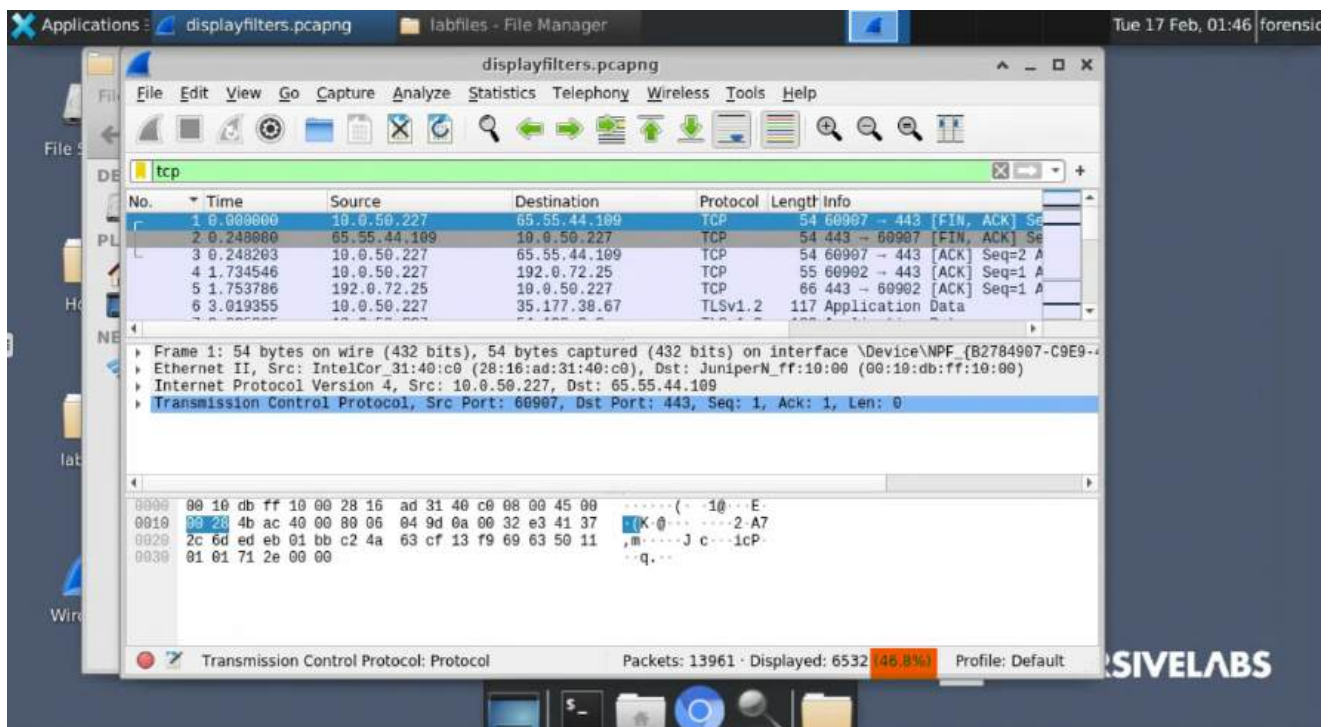
- **Filter by Protocol:** Simply type the protocol name.
 - `tcp` (Shows all TCP packets)
 - `dns` (Shows all DNS packets)
- **Filter by Specific Value:** Target a specific port or address.
 - `tcp.port == 80` (Shows TCP traffic on port 80/HTTP)
- **Finding Supported Protocols:** View the full list via **View** → **Internals** → **Supported Protocols**.

2. Open the PCAP file located in the `labfiles` directory.



3. Analyze and identify the information needed to complete the lab exercise.

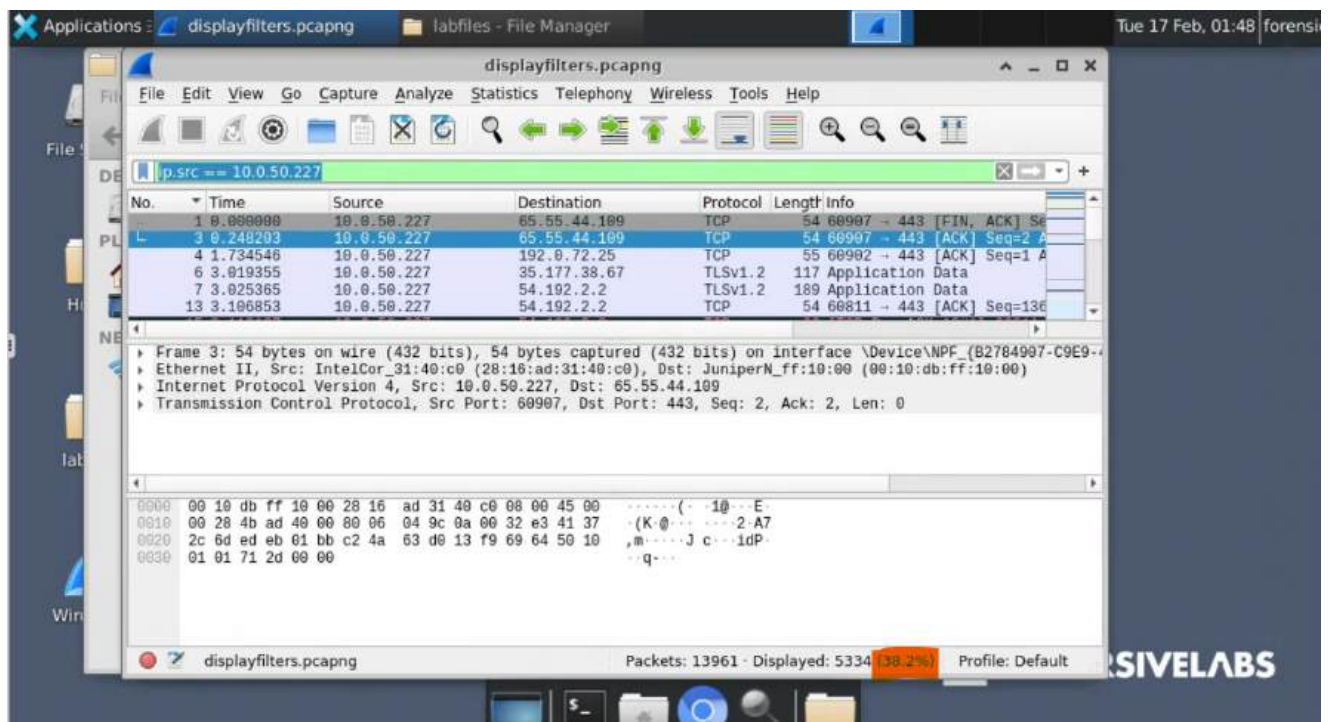
4. From the PCAP provided, apply a filter to display all TCP packets. What percentage of results are then displayed in the current capture?



Answer: 46.8%

- From the PCAP provided, apply a filter to display all packets from the source IP 10.0.50.227. What percentage of results are then displayed in the current capture?

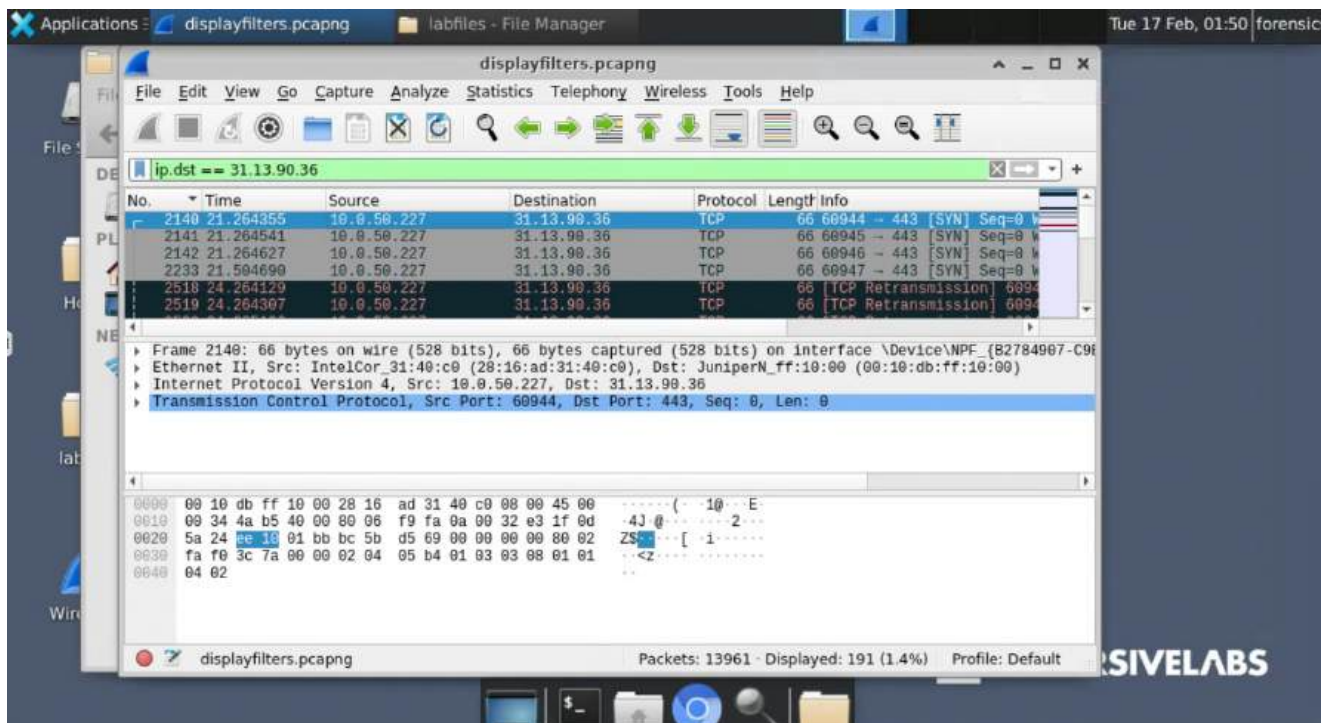
ip.src == 10.0.50.227



Answer: 38.2%

6. From the PCAP provided, apply a filter to display all packets sent to the IP 31.13.90.36. What is the source IP of each of these?

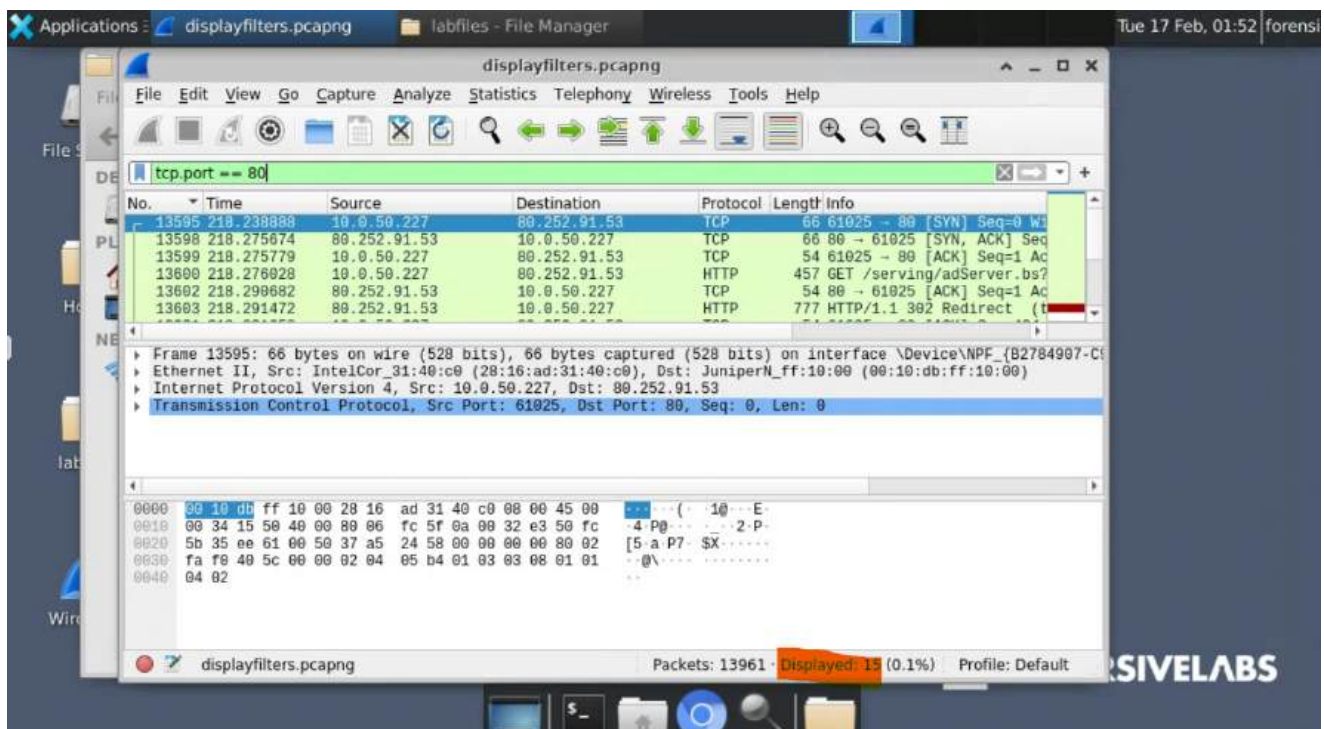
`ip.dst == 31.13.90.36`



Answer: 10.0.50.227

7. From the PCAP provided, apply a filter to display all TCP packets on port 80. How many packets are displayed?

`tcp.port == 80`



Answer: 15

8. `ip.dst == 10.10.10.10`. What does this line of code do?

- A. Filter capture to show all packets sent to the IP address 10.10.10.10
- B. Filter capture to show everything but the packets sent to 10.10.10.10
- C. Filter capture to hide all packets sent to the IP address 10.10.10.10

Answer: A

Display Filters - Diving In

1. Read the **Briefing** panel to understand some more complex filters/operators.

Visual Feedback

- **Red Bar:** The filter expression is invalid/unaccepted.
- **Green Bar:** The expression is valid and will work as expected.
- **Yellow Bar:** The expression is accepted but may not work as intended.

Advanced Operators

1. Comparing Values

- **Basic Comparisons:** Use operators like `eq`, `ne`, `gt`, `lt` to compare field values.
- **contains** : Searches for a **sequence of characters** (string) anywhere within a field.
 - *Example:* `http contains "https://malicious.com"`
- **matches (or ~)** : Uses **Perl-compatible regular expressions (PCRE)** for pattern matching. Allows for case-sensitive matching.
 - *Example:* `tcp.flags.str matches "(?-i)ACK"`

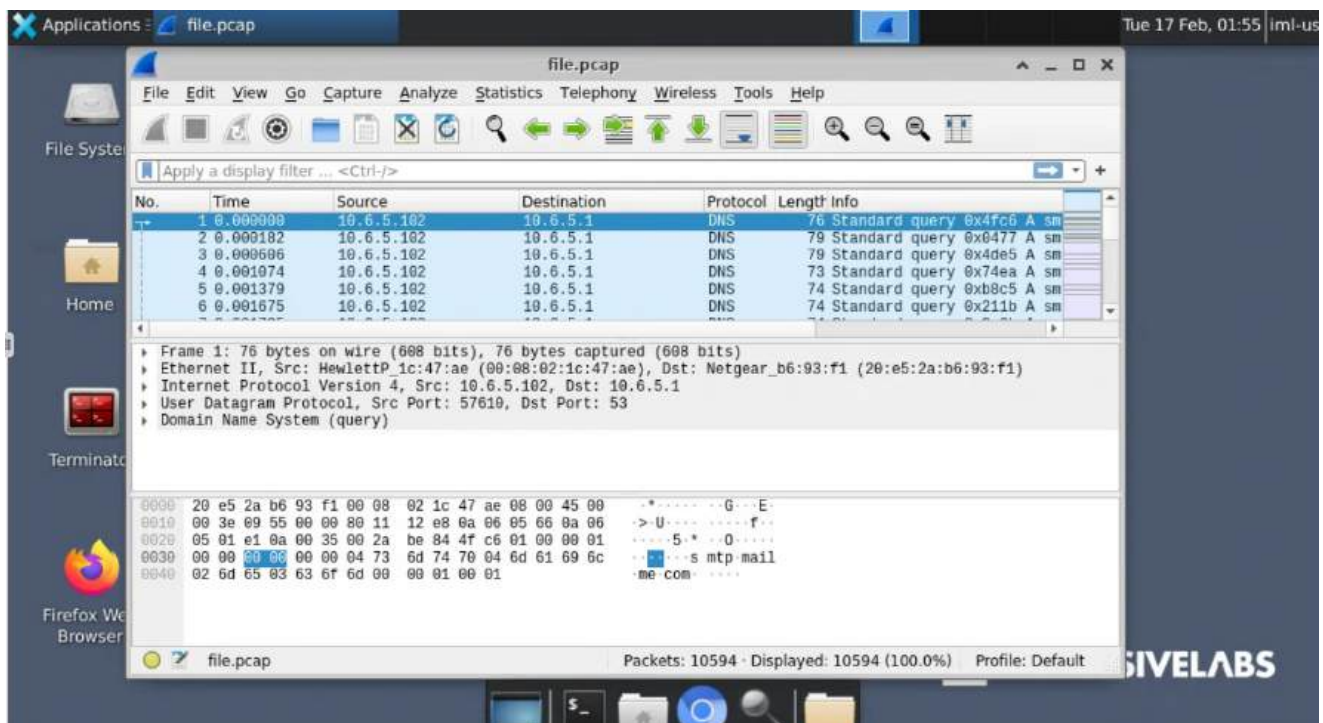
2. The Slice Operator (`[]`)

- **Purpose:** Filters based on a **specific portion (range)** of a field (e.g., first 3 bytes of a MAC address).
- **Syntax:** `field[start:length]`
 - *Example:* `eth.src[0:3] == 32.65.76` (Filters for Ethernet addresses starting with that specific byte sequence).
- **Negative Offsets:** Count from the end of the field.
 - *Example:* `eth.src[-3:3]` looks at the last three bytes.
- **Concatenation:** Combine multiple slices using a comma (`,`).

3. The Membership Operator (`in`)

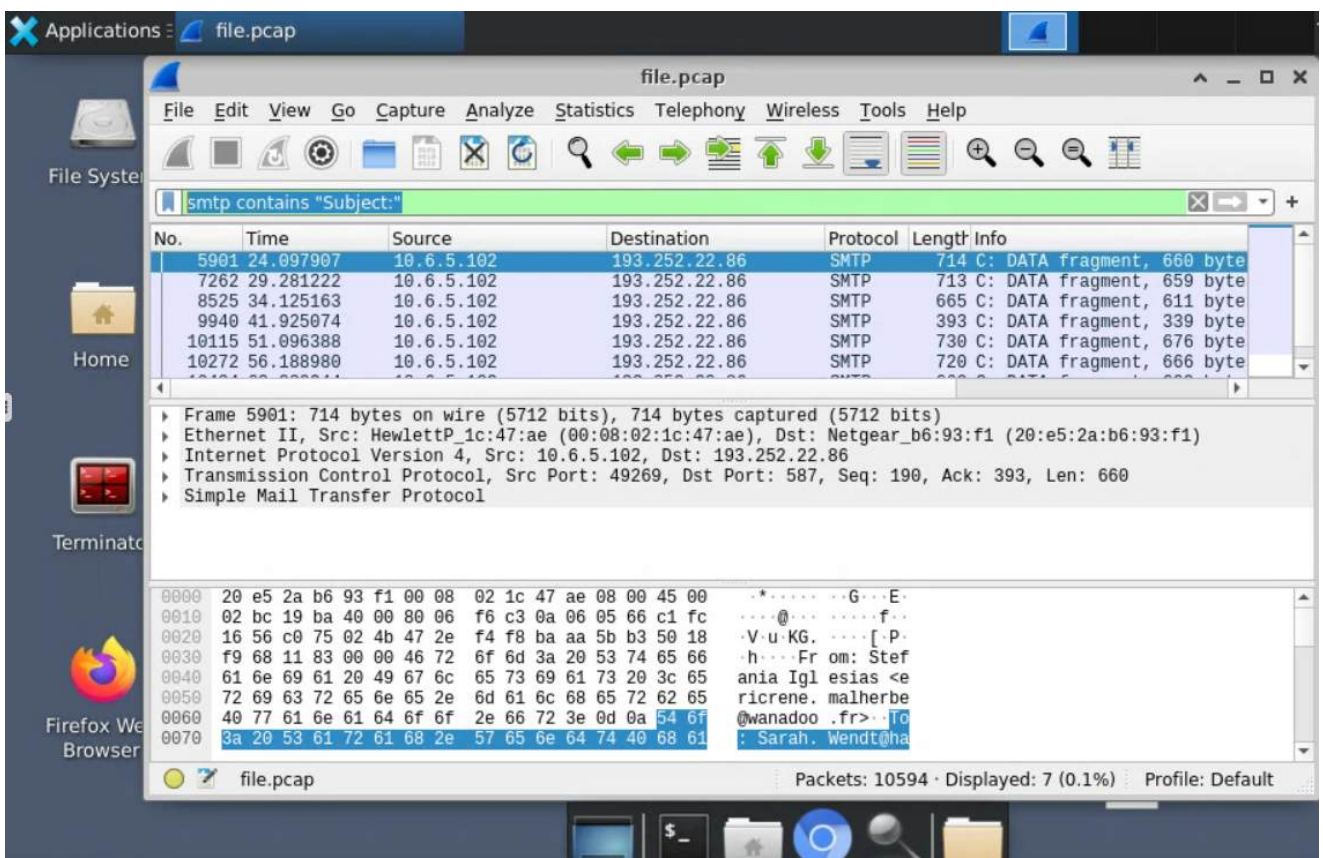
- **Purpose:** Tests if a field matches **any value** within a specified set.
 - **Syntax:** `field in {value1 value2 value3}`
 - *Example:* `tcp.srcport in {80 8080 808080}` (Shows traffic from any of these source ports).
 - **Ranges:** Can include ranges within the set using `..`.
 - *Example:* `tcp.srcport in {80 8080 80..8080}`
-

2. Navigate to your **Desktop** and open **file.pcap** in Wireshark.

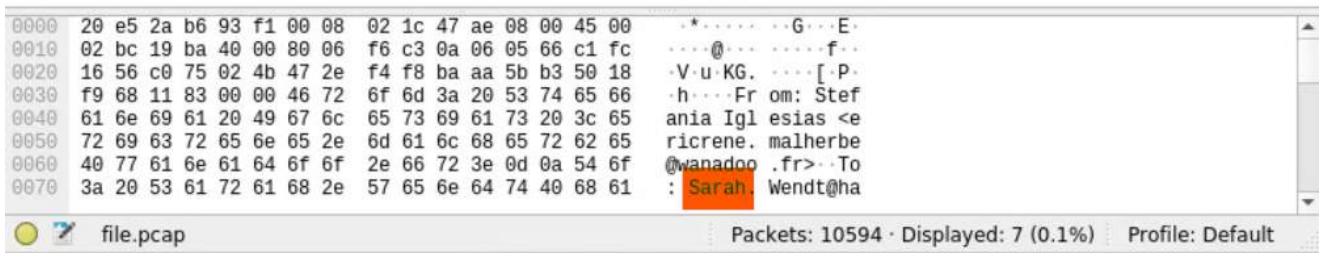


3. Apply a filter that displays all SMTP traffic containing the text "Subject: "

smtp contains "Subject:"



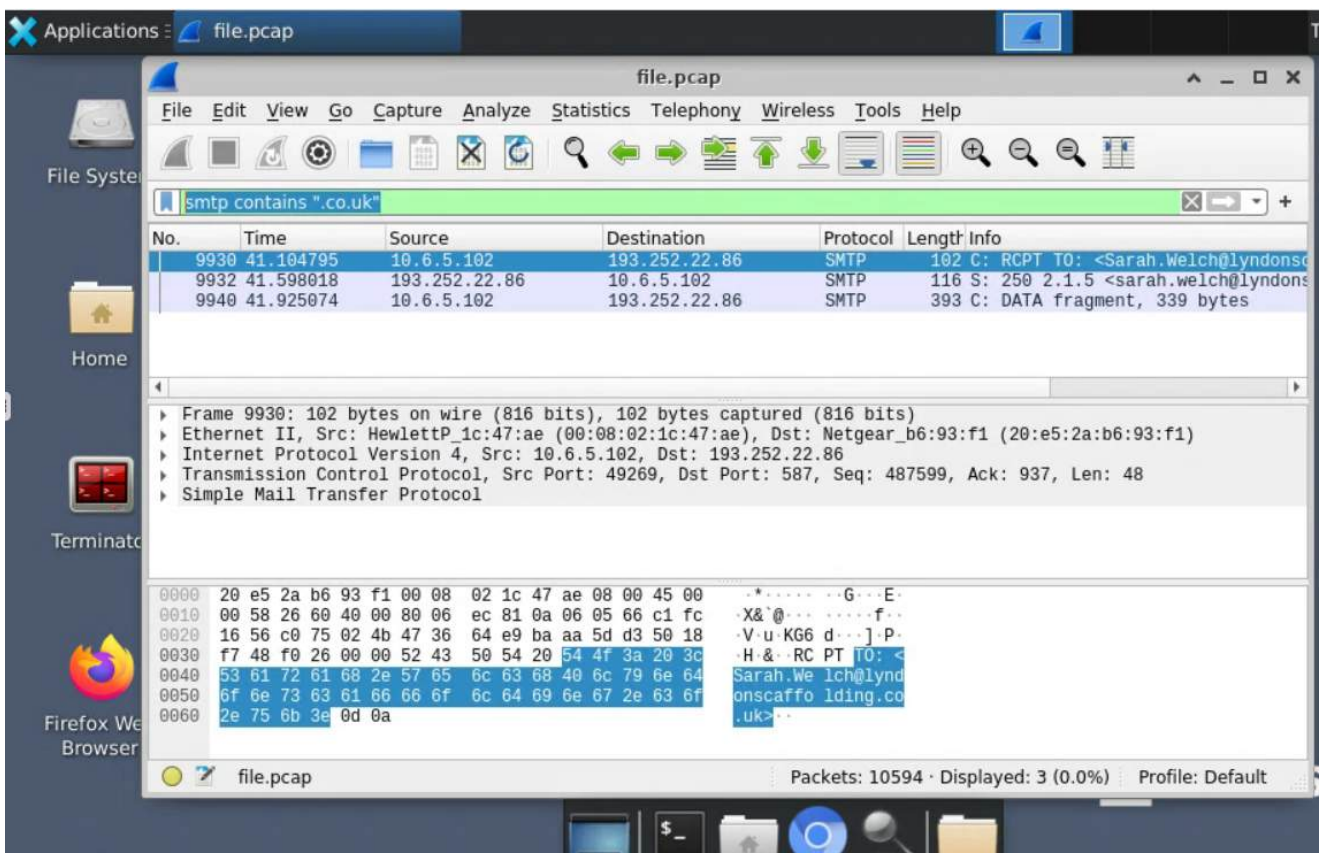
4. What is the first name of the recipient of that email?



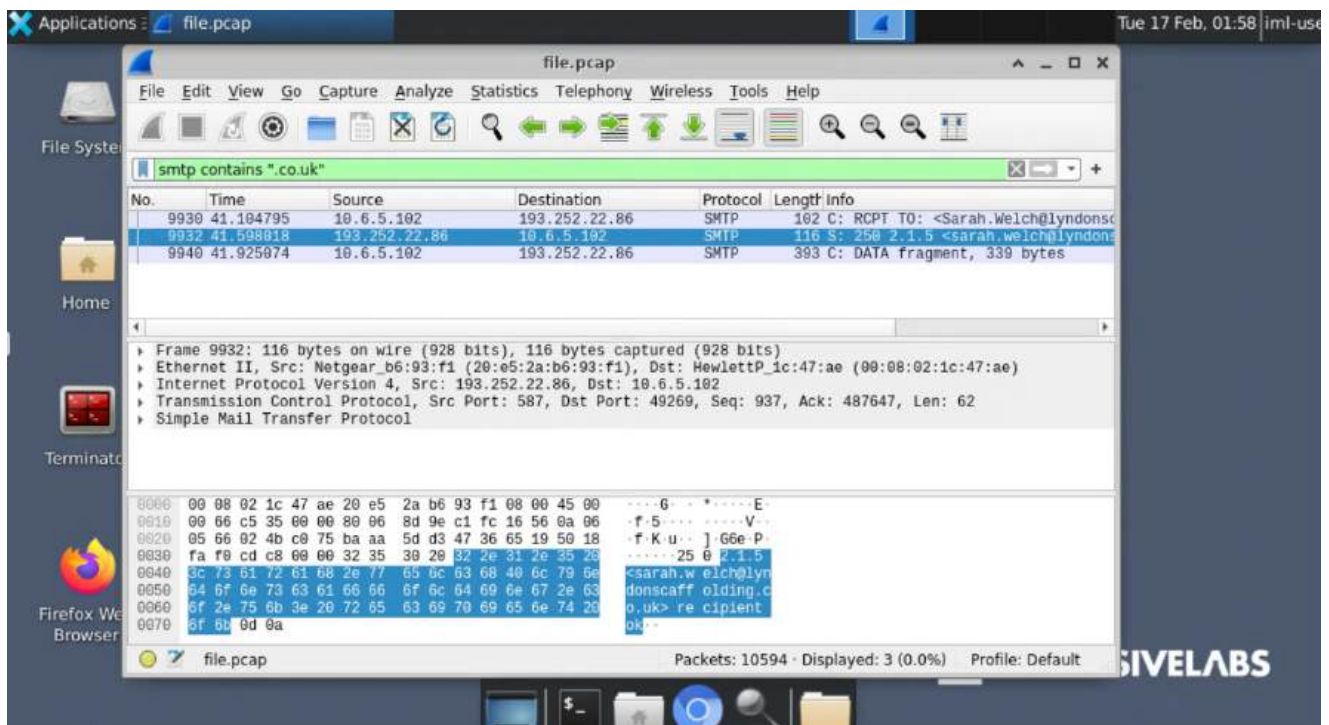
Answer: Sarah

5. Change the filter so it now displays all SMTP response traffic matching the text ".co.uk".

smtp contains ".co.uk"



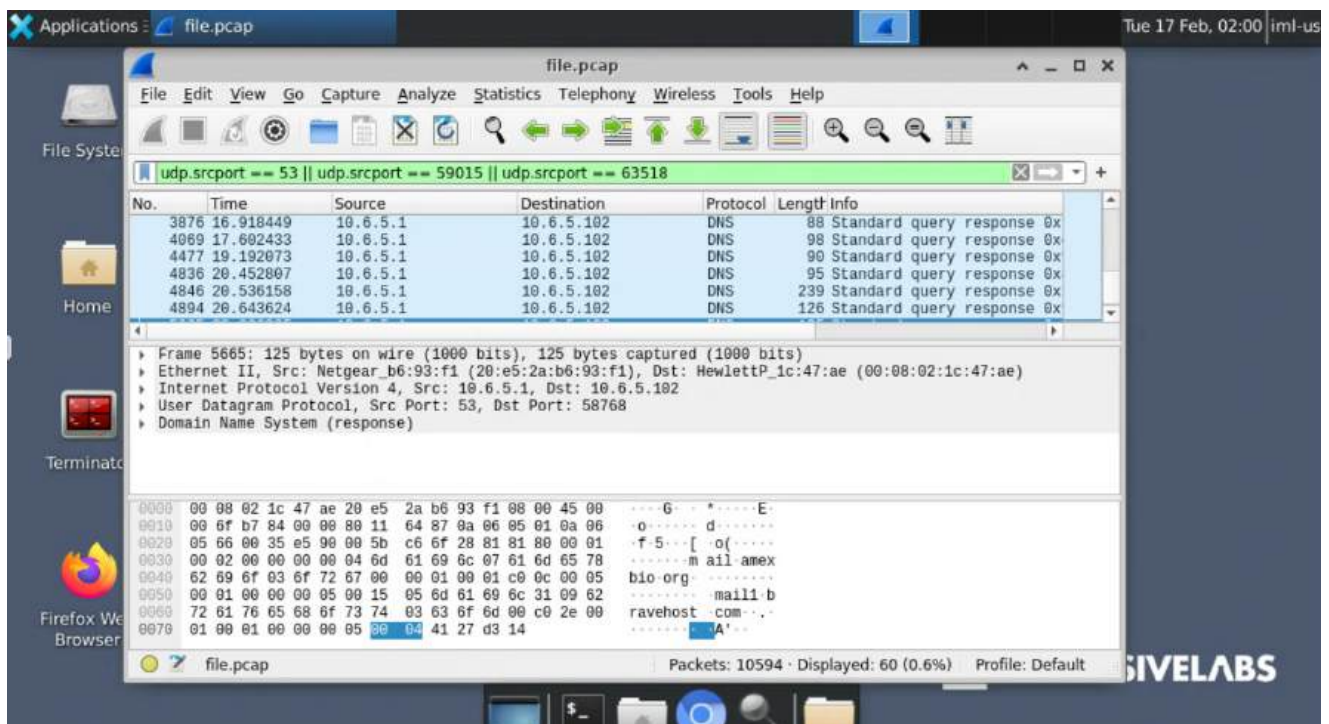
6. What is the frame number of this packet?



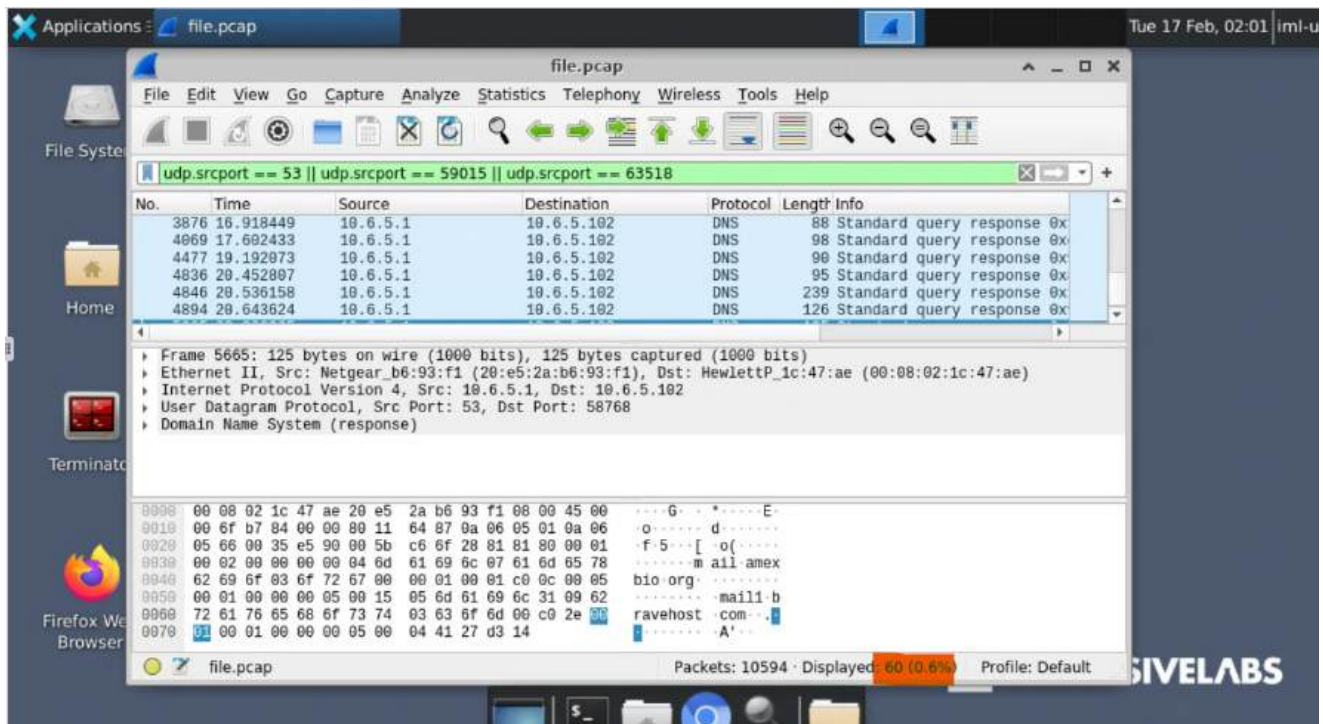
Answer: 9932

- Remove the existing filter. Now, apply a filter that displays all packets from UDP source ports 53, 59015, and 63518.

`udp.srcport == 53 || udp.srcport == 59015 || udp.srcport == 63518`



- How many packets are then displayed?



Answer: 60

9. Take the following slice expression `(frame[-4:4] == 0.1.2.3)`.

`frame[-4:4] == 00:01:02:03`

10. At which offset does the slice begin?

Answer: -4

11. Take the following slice expression `(frame[:4] == 0.1.2.3)`.

`frame[:4] == 00:01:02:03`

12. At which offset does the slice begin?

Answer: 0

Display Filters - Combining Filters

Purpose of Combining Filters

- **Why:** In large packet captures, single filters are often not enough. Combining filters allows you to narrow down traffic using multiple conditions in one expression.
- **Goal:** To create precise searches for specific IOCs or traffic patterns (e.g., a specific IP communicating on a specific port).

Logical Operators

These are used to join multiple filter expressions:

Operator	Meaning	Example
and / &&	Both conditions must be true	<code>ip.src == 192.168.0.1 && tcp.port == 80</code>
or / \	Either condition can be true	<code>ip.src == 192.168.0.1 \ tcp.port == 80</code>
not / !	Excludes packets matching the condition	<code>!(tcp.port == 80)</code>

Example Breakdown

Filter: `ip.src == 192.168.0.1 && tcp.port == 80`

- `ip.src == 192.168.0.1` : Looks for packets where the source IP address is 192.168.0.1.
- `&&` : The logical AND operator, meaning **both** conditions must be met.
- `tcp.port == 80` : Looks for packets with TCP port 80 (HTTP).

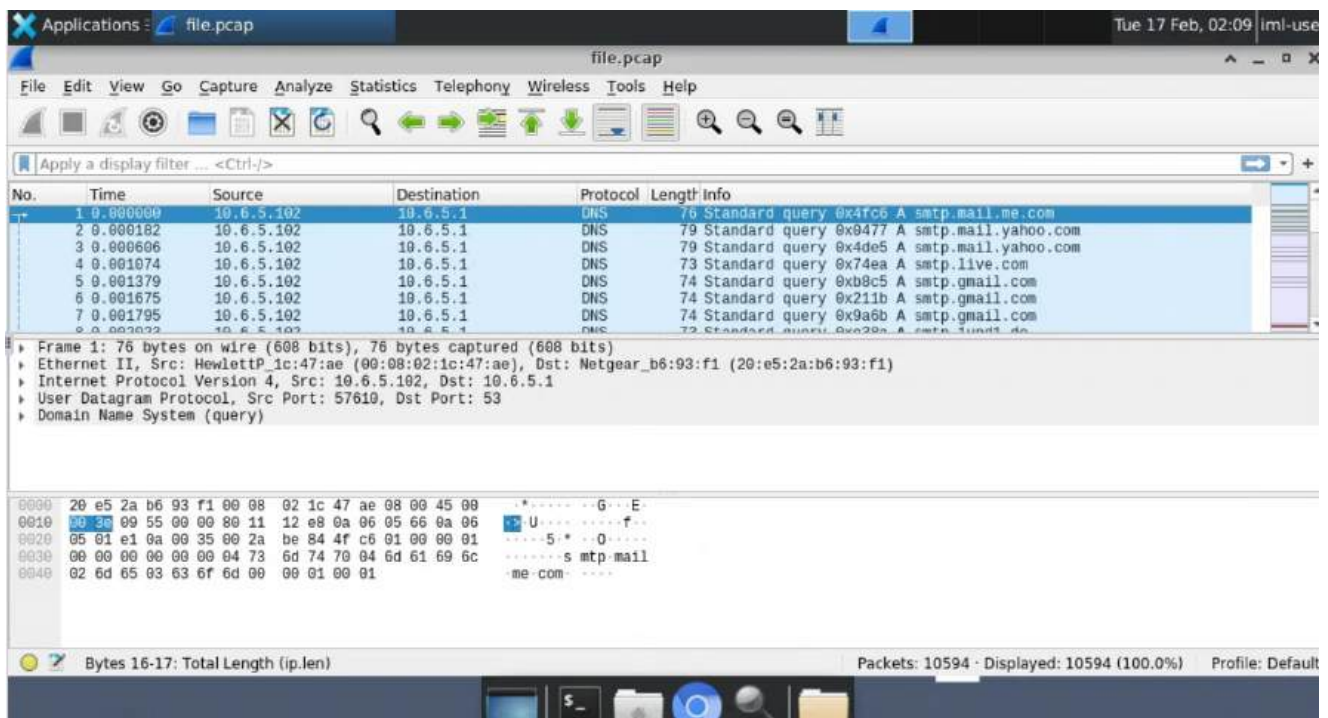
Result: Shows only packets coming from IP `192.168.0.1` **AND** using TCP port 80.

Exclusion/ Negation

To filter **for** an IP but **exclude** a specific port:

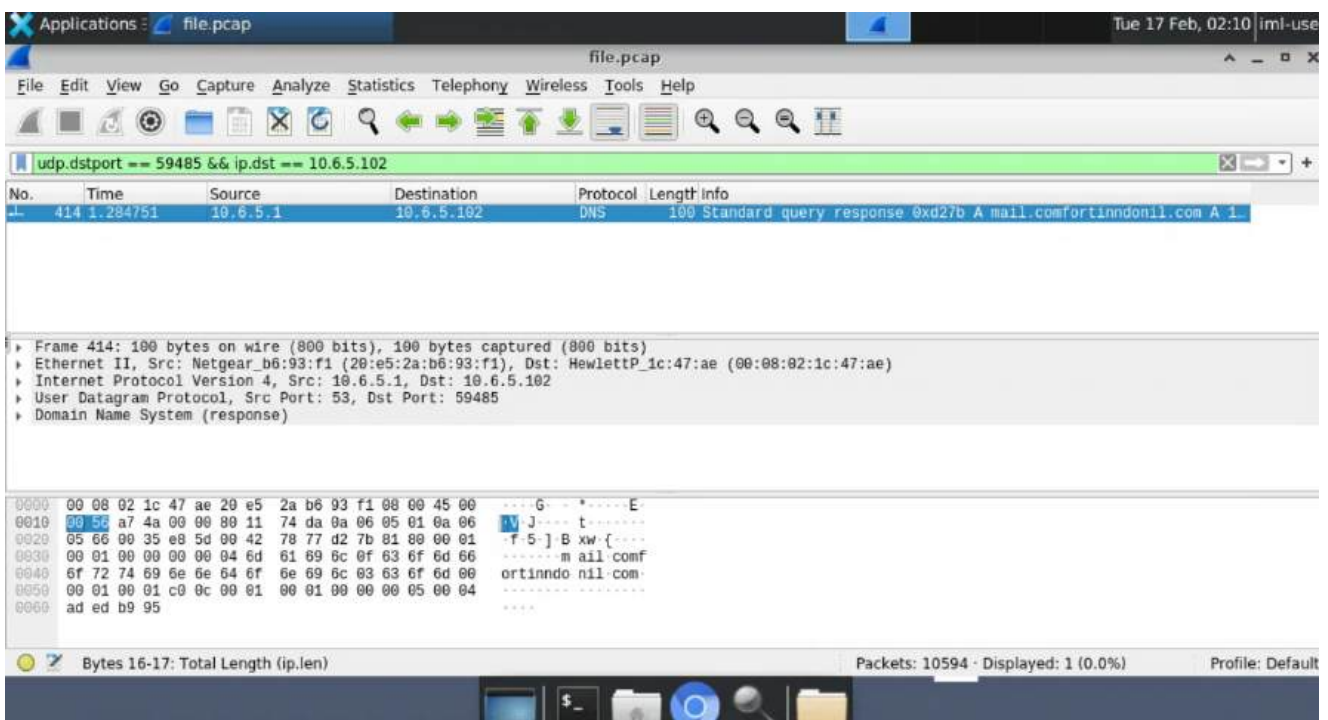
- **Method 1:** Use the `≠` operator.
 - `ip.src == 192.168.0.1 && tcp.port ≠ 80`
- **Method 2 (More Reliable):** Use brackets `()` with the `!` operator.
 - `ip.src == 192.168.0.1 && !(tcp.port == 80)`

-
1. Open the PCAP located on your desktop in Wireshark.

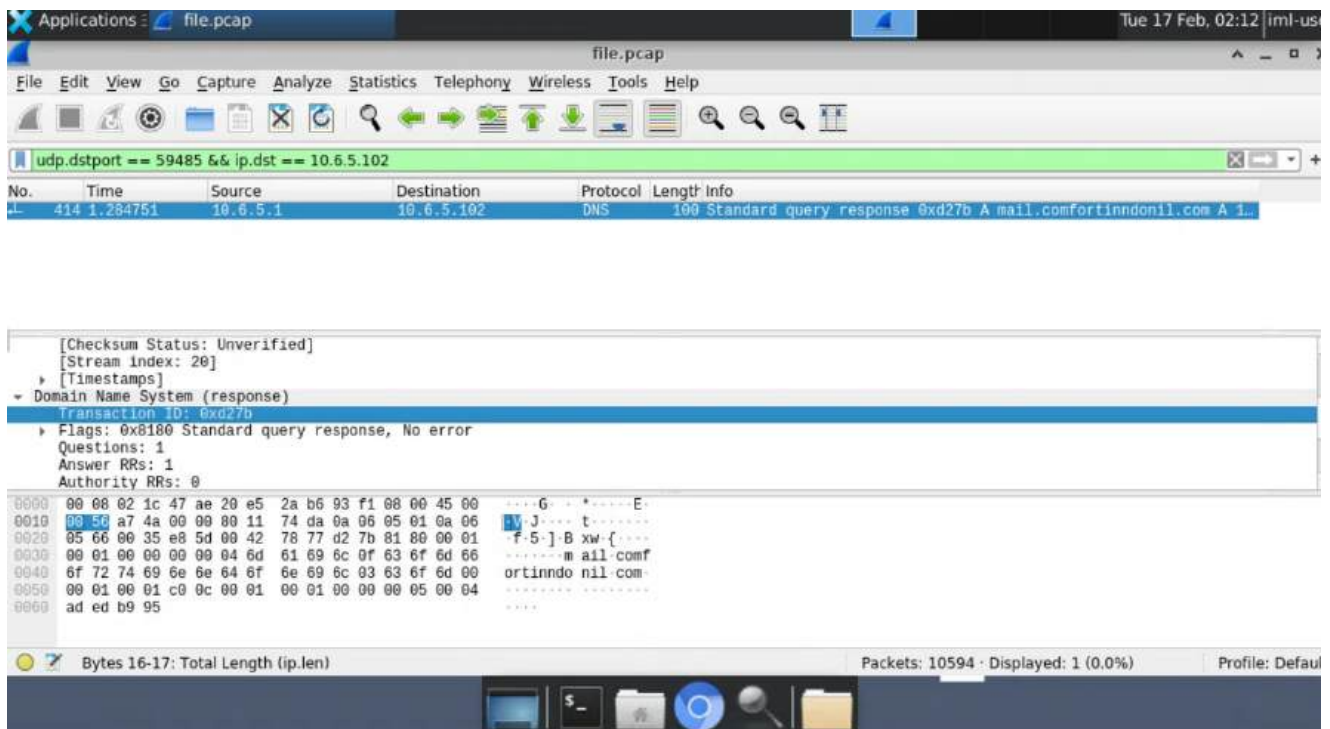


- From the PCAP provided, apply a filter to display all packets where the destination UDP port is **59485** and the destination IP address is **10.6.5.102**.

udp.dstport == 59485 && ip.dst == 10.6.5.102

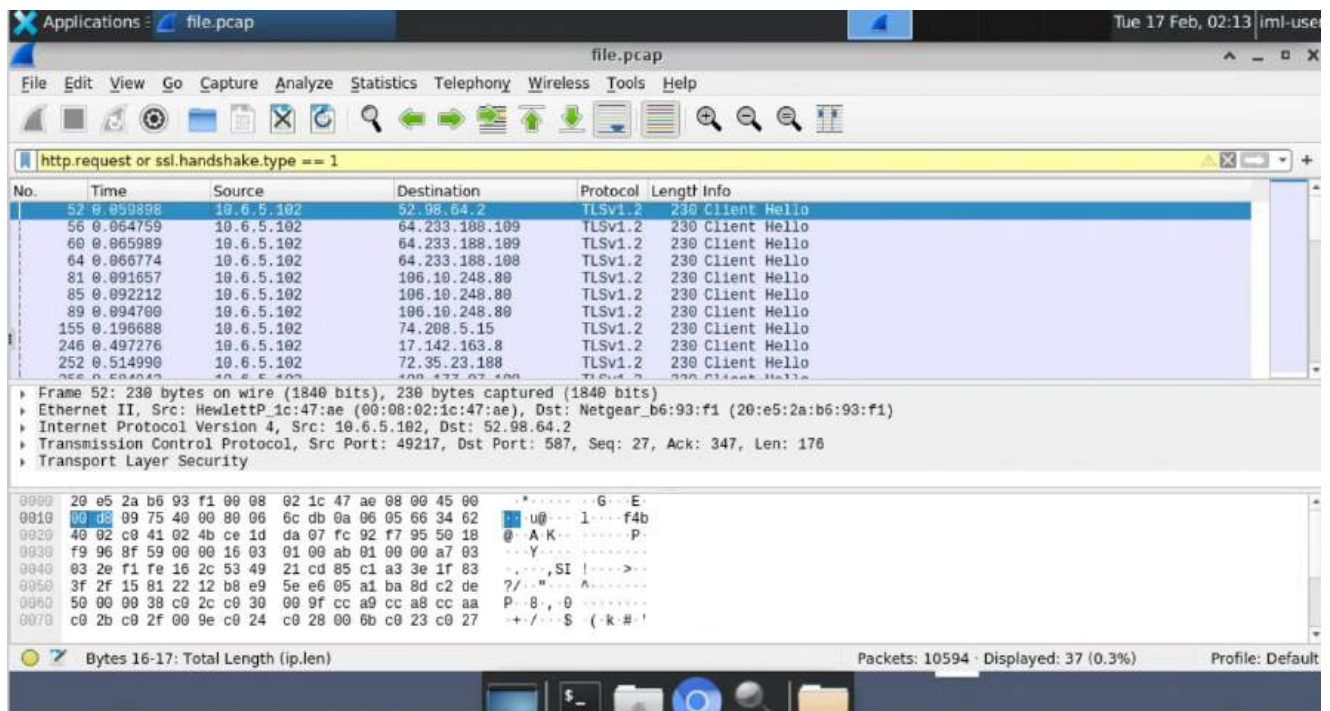


- What is the transaction ID of this packet response?



Answer: 0xd27b

4. From the PCAP provided, apply the following filter to display all web traffic:
- ```
http.request or ssl.handshake.type == 1
```

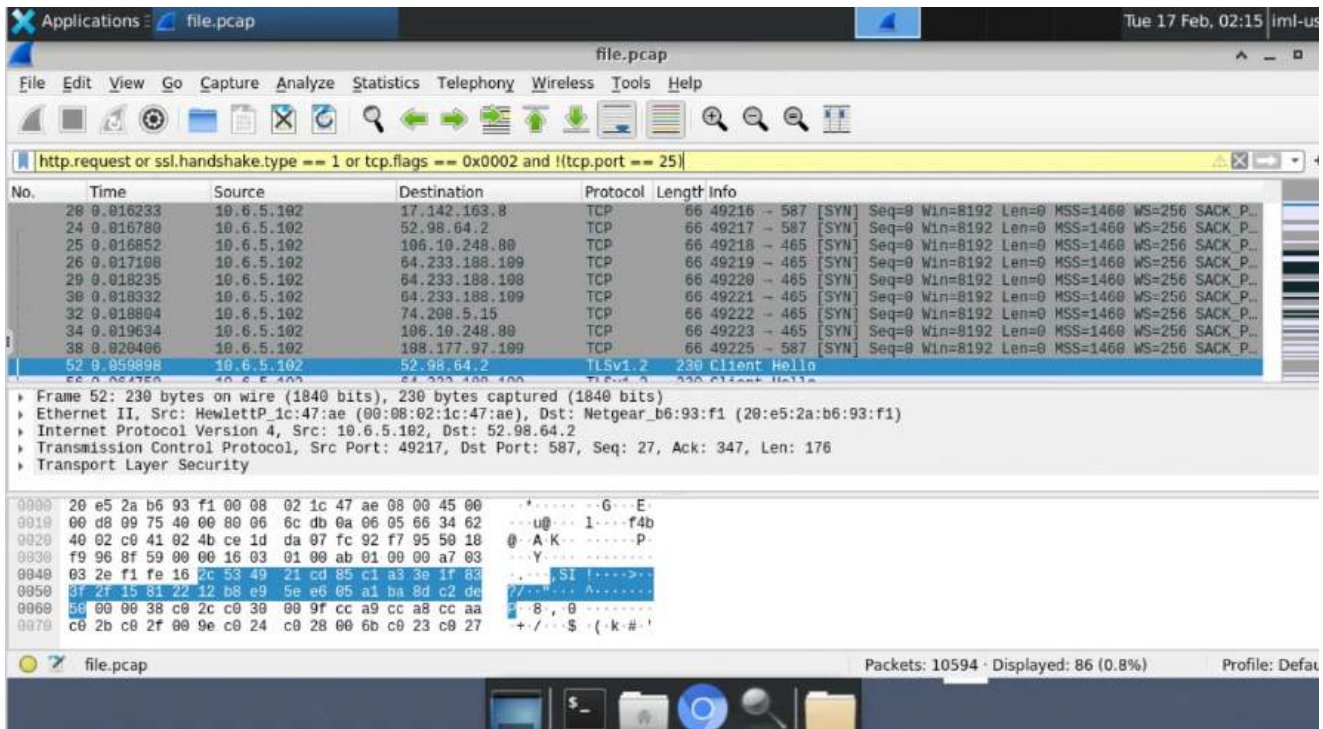


5. What percentage of results are then displayed in the capture?

Answer: 0.3%

6. Take the filter used in the previous question, and add an OR expression that includes packets where `tcp.flags` is equal to `0x0002`. Add another expression to the filter which excludes packets going to/from TCP port 25.

`http.request or ssl.handshake.type == 1 or tcp.flags == 0x0002 and !(tcp.port == 25)`

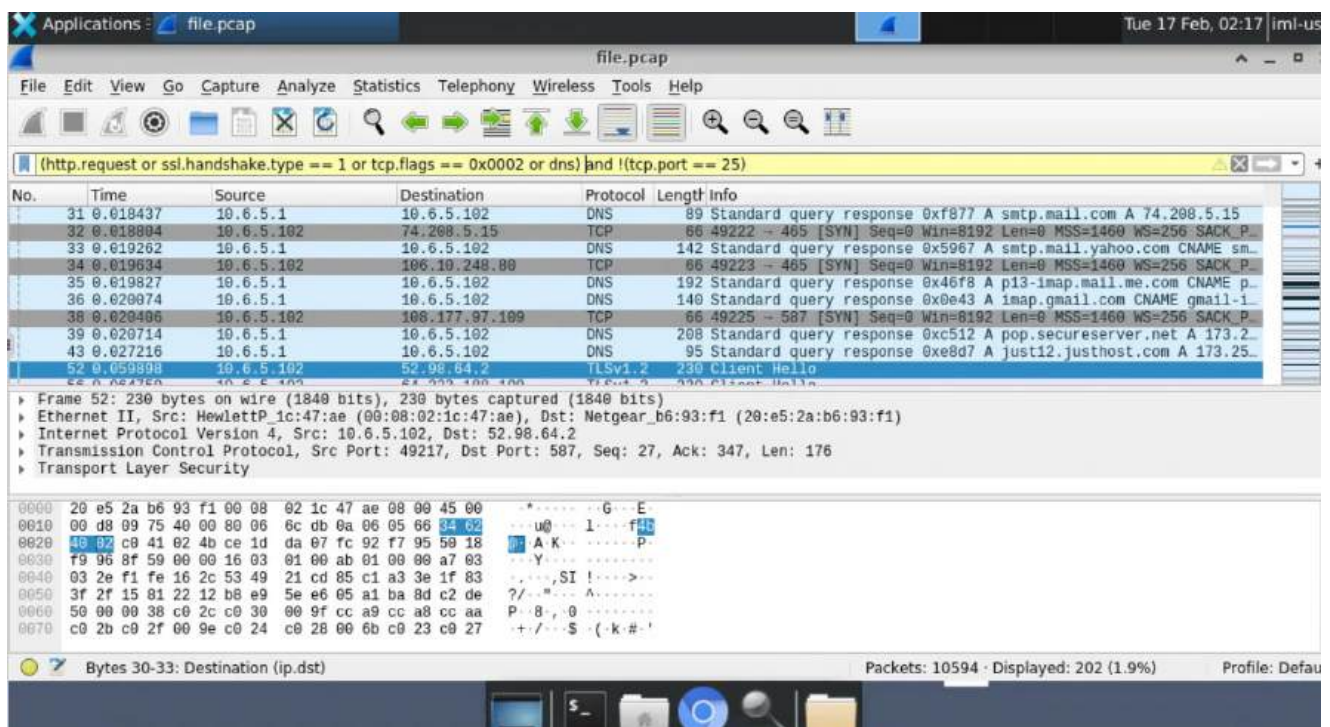


7. What percentage of results are then displayed in the capture?

Answer: 0.8%

8. Using the same filter, expand the OR section to also include DNS packets.

`(http.request or ssl.handshake.type == 1 or tcp.flags == 0x0002 or dns) and !(tcp.port == 25)`



9. What percentage of results are now displayed in the capture?

Answer: 1.9%

## Metrics and Statistics

### Purpose of Statistics

- **Why:** Manually filtering for IPs assumes you already know what to look for. Statistics provide a **bird's-eye view** of the capture to identify anomalies, top talkers, and patterns quickly.
- **Goal:** To surface key investigative leads (like large data transfers or suspicious endpoints) without prior knowledge of the traffic.

### Key Statistics in Wireshark

Found under the **Statistics** menu at the top of the screen.

#### 1. Conversations

- **Definition:** Traffic between **two specific endpoints** (e.g., all traffic between IP A and IP B).

- **Use Case:** Identify which pairs of devices are communicating the most. Sorting by "Bytes" can reveal potential data exfiltration or command & control traffic.
- **Organization:** Grouped by protocol (IPv4, TCP, UDP, etc.). The number in the tab indicates how many conversations exist for that protocol.

## 2. Endpoints

- **Definition:** A single logical device on the network (e.g., one specific IP address).
- **Difference from Conversations:** Shows traffic **to and from** a single machine, rather than a pair.
- **Use Case:** Identify which specific device is the "noisiest" or is sending/receiving the most data.

## 3. Resolved Addresses

- **Definition:** A list of all IP addresses and their corresponding **hostnames** (resolved from DNS traffic in the capture).
- **Use Case:** Quickly get a human-readable summary of all external resources the infected host contacted (e.g., `malware.com` instead of `192.0.2.1`). Useful for reports.

## 4. IO Graphs

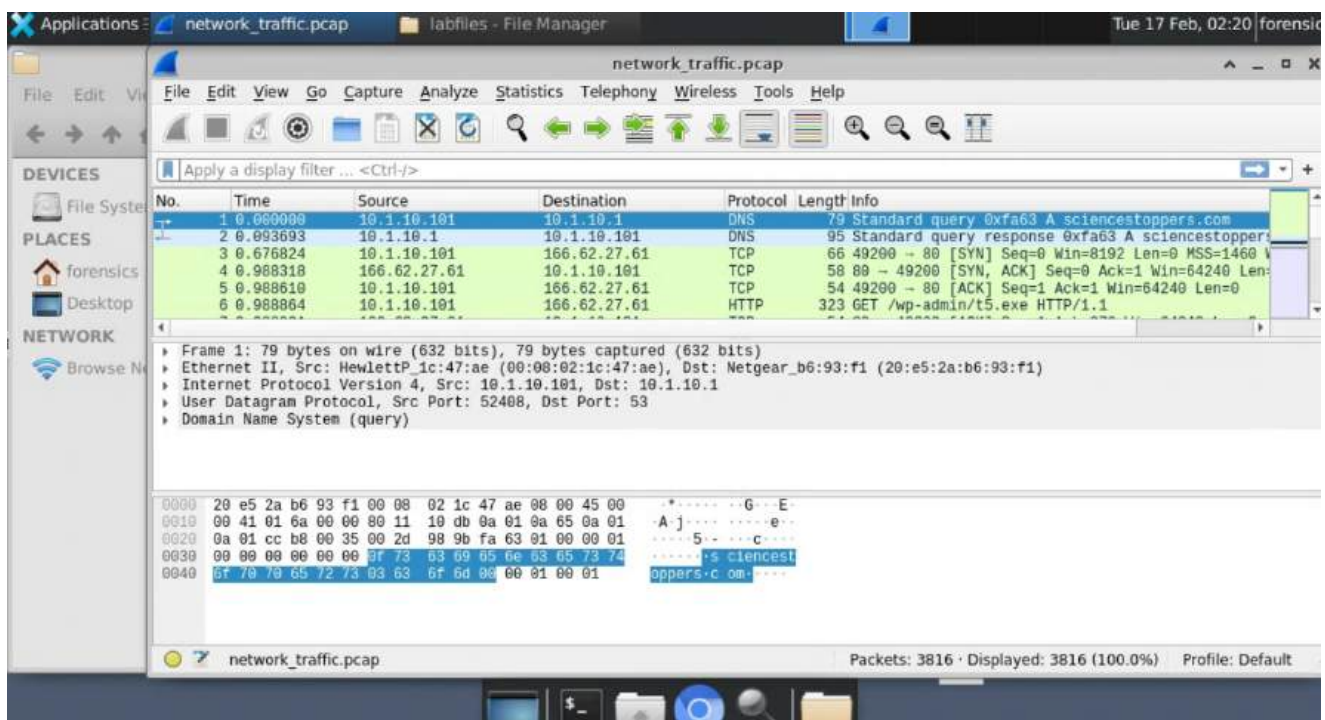
- **Definition:** A **visual, graphical representation** of traffic over time.
- **Use Case:** Spot patterns visually (e.g., periodic beaconing to a C2 server). You can plot up to five different filters on the same graph to compare protocols or traffic types.
- **Export:** Can be saved as an image or copied as CSV for reports.

## Summary Diagram

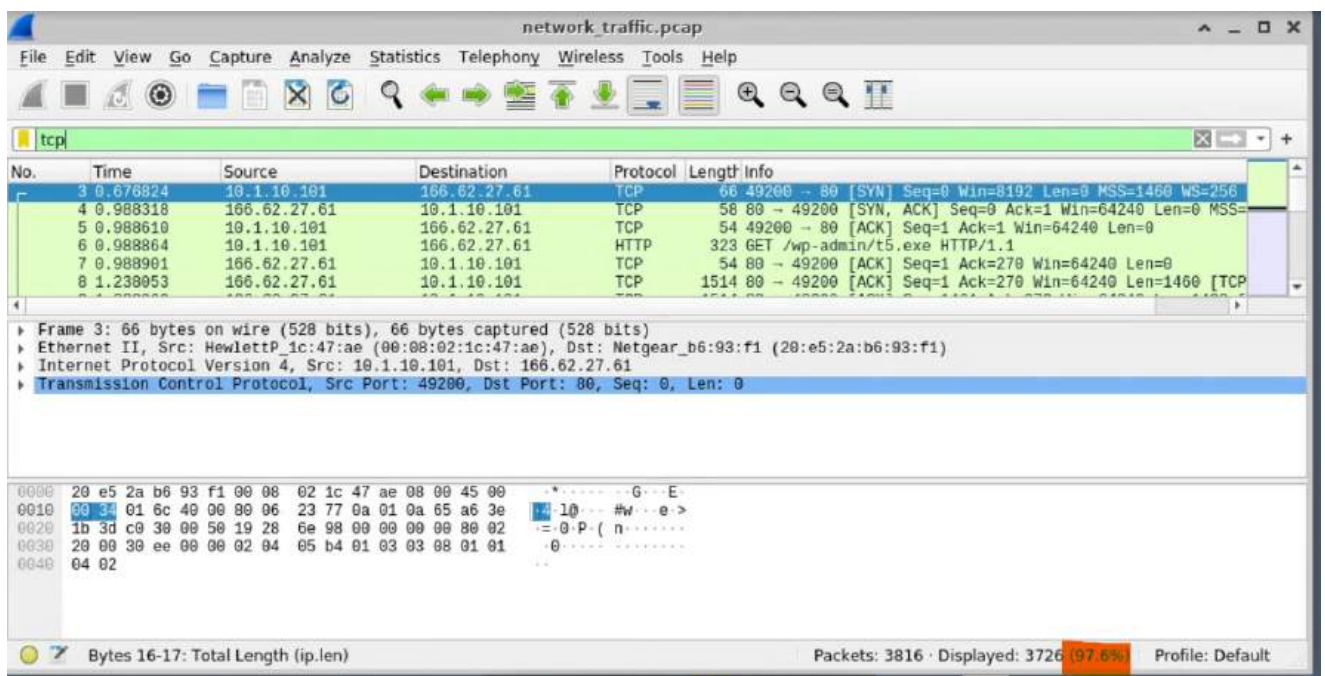
- **Conversations:** Show traffic *between* pairs.
- **Endpoints:** Show traffic *to/from* a single machine.
- Both display **byte counts** to help spot unusual transfers.

- 
1. Open the PCAP file located in the `labfiles` directory and use Wireshark statistics to analyze the packet capture.





2. What percentage of packets in the capture file are TCP? (rounded to one decimal place)

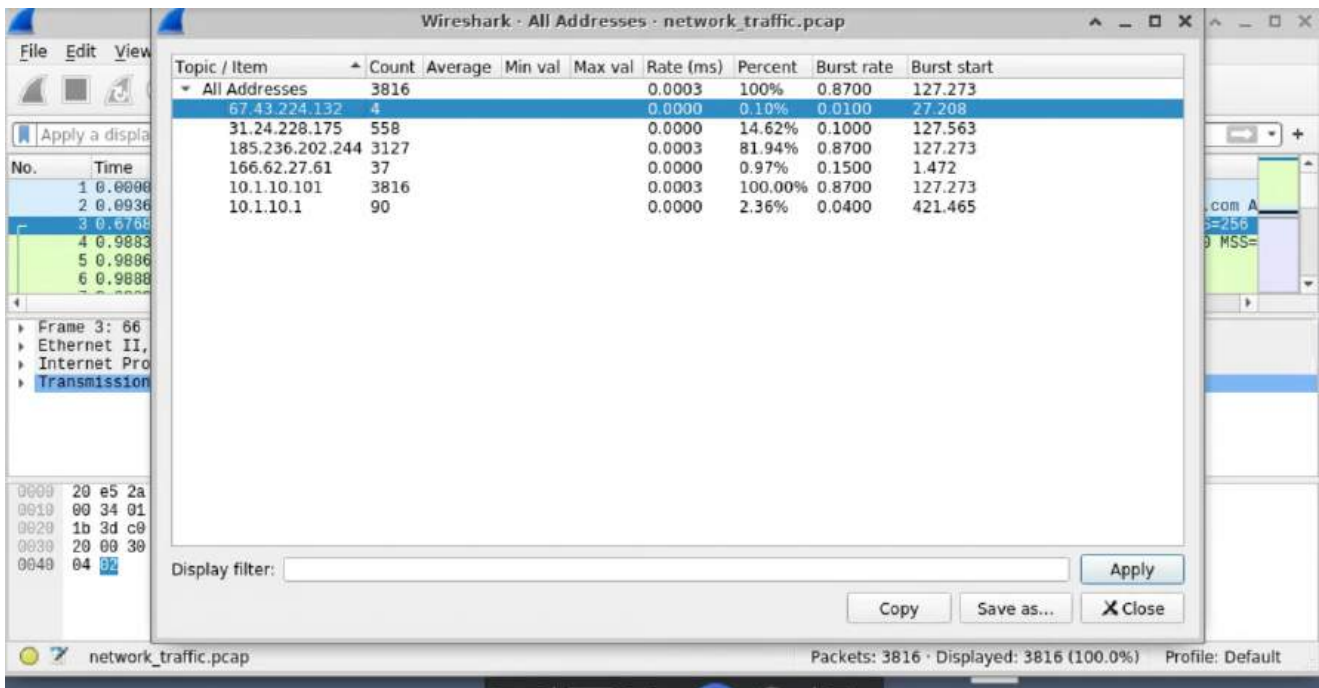


Answer: 97.6%

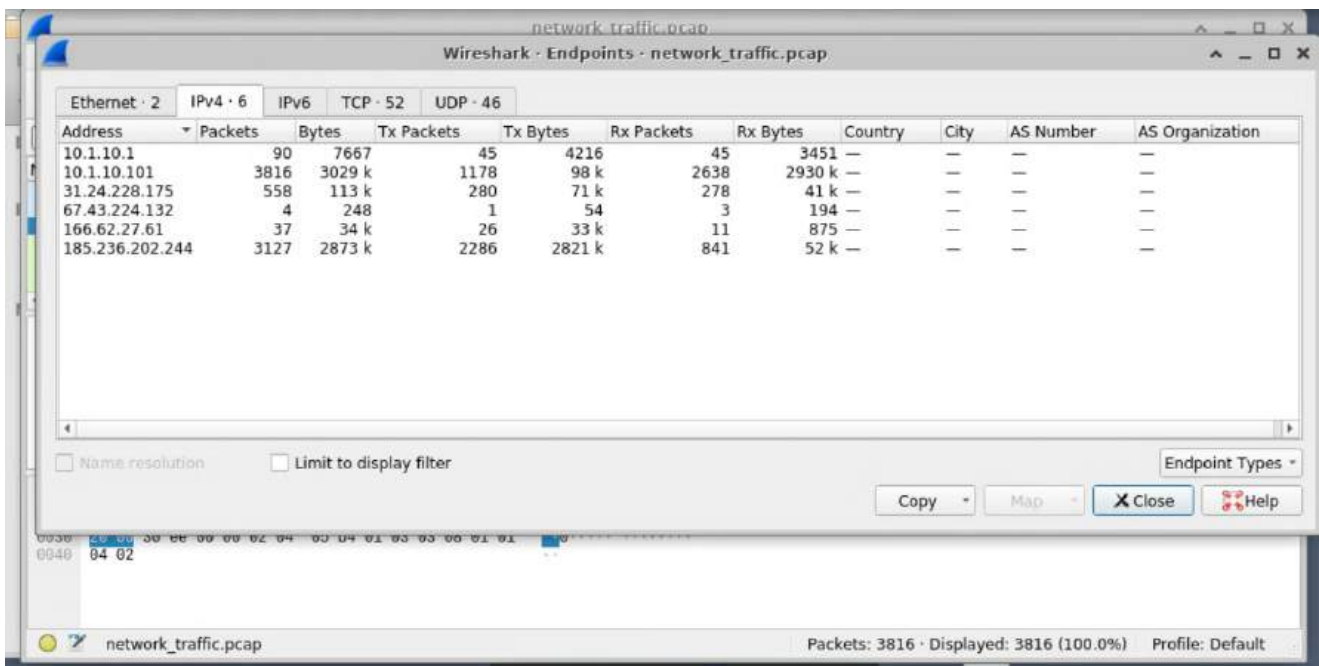
3. How many IPv4 'Endpoints' are in the capture file?

- Navigate to the **Statistics** menu at the top of the window.
- Select **Endpoints** from the dropdown list.

- In the window that pops up, click on the **IPv4** tab.



or



Answer: 6

4. How many IPv4 'Conversations' are in the capture file?

**Statistics -> Conversations -> IPv4**

network\_traffic.pcap

Wireshark · Conversations · network\_traffic.pcap

Ethernet · 1IPv4 · 5IPv6TCP · 47UDP · 45

| Address A   | Address B       | Packets | Bytes  | Packets A → B | Bytes A → B | Packets B → A | Bytes B → A | Rel Start  | Duration   | Bit |
|-------------|-----------------|---------|--------|---------------|-------------|---------------|-------------|------------|------------|-----|
| 10.1.10.1   | 10.1.10.101     | 90      | 7667   | 45            | 4216        | 45            | 3451        | 0.000000   | 11255.8799 |     |
| 10.1.10.101 | 166.62.27.61    | 37      | 34 k   | 11            | 875         | 26            | 33 k        | 0.676824   | 15.6102    |     |
| 10.1.10.101 | 67.43.224.132   | 4       | 248    | 3             | 194         | 1             | 54          | 27.208321  | 75.2649    |     |
| 10.1.10.101 | 185.236.202.244 | 3127    | 2873 k | 841           | 52 k        | 2286          | 2821 k      | 53.624262  | 11157.0585 |     |
| 10.1.10.101 | 31.24.228.175   | 558     | 113 k  | 278           | 41 k        | 280           | 71 k        | 122.832877 | 11133.7982 |     |

☐ Name resolution☐ Limit to display filter☐ Absolute start time

Conversation Types ▾

Copy ▾Follow Stream...Graph...CloseHelp

network\_traffic.pcapPackets: 3816 · Displayed: 3816 (100.0%)Profile: Default

Answer: 5

5. How many packets are sent between the IP addresses 10.1.10.101 and 185.236.202.244?

network\_traffic.pcap

Wireshark · Conversations · network\_traffic.pcap

Ethernet · 1

IPv4 · 5

IPv6

TCP · 47

UDP · 45

| Address A   | Address B       | Packets | Bytes  | Packets A → B | Bytes A → B | Packets B → A | Bytes B → A | Rel Start  | Duration   | Bit |
|-------------|-----------------|---------|--------|---------------|-------------|---------------|-------------|------------|------------|-----|
| 10.1.10.1   | 10.1.10.101     | 90      | 7667   | 45            | 4216        | 45            | 3451        | 0.000000   | 11255.8799 |     |
| 10.1.10.101 | 166.62.27.61    | 37      | 34 k   | 11            | 875         | 26            | 33 k        | 0.676824   | 15.6102    |     |
| 10.1.10.101 | 67.43.224.132   | 4       | 248    | 3             | 194         | 1             | 54          | 27.208321  | 75.2649    |     |
| 10.1.10.101 | 185.236.202.244 | 3127    | 2873 k | 841           | 52 k        | 2286          | 2821 k      | 53.624262  | 11157.0585 |     |
| 10.1.10.101 | 31.24.228.175   | 558     | 113 k  | 278           | 41 k        | 280           | 71 k        | 122.832877 | 11133.7982 |     |

☐ Name resolution

☐ Limit to display filter

☐ Absolute start time

Conversation Types ▾

Copy ▾

Follow Stream...

Graph...

X Close

Help

network\_traffic.pcap

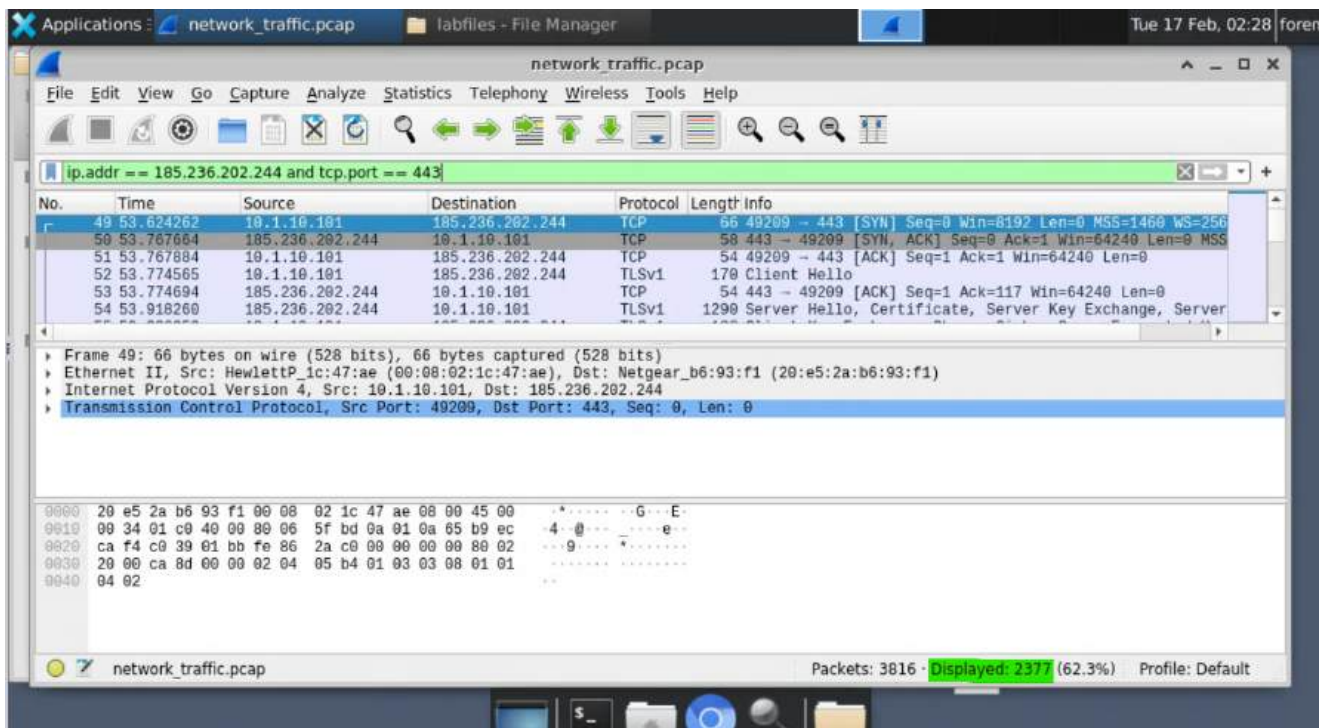
Packets: 3816 · Displayed: 3816 (100.0%) Profile: Default

Answer: 3127

6. How many packets are sent AND received from 185.236.202.244 port 443?

Apply the filter:

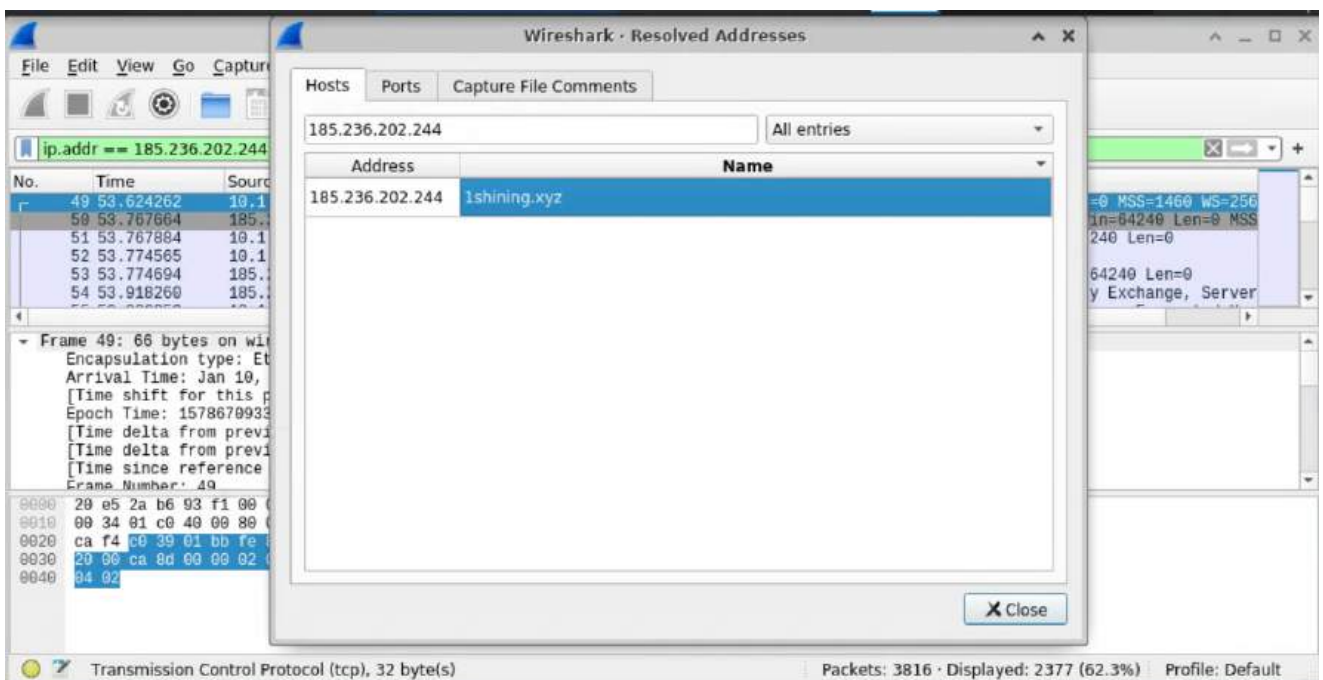
ip.addr == 185.236.202.244 and tcp.port == 443



Answer: 2377

7. What is the resolved host of the IP address 185.236.202.244?

- Go to **Statistics -> Resolved Addresses**.
- In the dialog box, click the **Hosts** tab.
- Use the search/filter bar at the bottom to type 185.236.202.244.
- The hostname will appear in the column next to the IP.





## Stream/Object Extraction

---

### Purpose

- **Why:** To move beyond raw packets and view or recover the actual **data being transmitted** (e.g., files, images, text) as seen by the application layer.
- **Goal:** To extract artifacts (objects) or reconstruct conversations for evidence and deeper inspection.

### Following Streams

- **What it does:** Reassembles the conversation between two endpoints into a single, readable view (ASCII or hex).
- **How to do it:** Right-click a packet → **Follow** → **Follow** <protocol> **Stream** (TCP, UDP, HTTP, TLS).
- **Benefit:** Simplifies analysis by showing the raw data exchange (e.g., the contents of a chat message or an FTP command) without packet overhead. You can also save the raw stream to disk.

### Exporting Objects

- **What it does:** Extracts complete files that were transferred over the network.
- **Supported protocols:** HTTP, SMB, etc.
- **How to do it:** **File** → **Export Objects** → Select protocol type (e.g., HTTP) → Choose file(s) to save.
- **Use Case:** Recovering malware, images, documents, or web pages transferred during the capture for offline analysis.

### Exporting Packet Bytes

- **What it does:** Exports the raw bytes from a **single specific packet** (or a selection).
- **When to use it:** When data is fragmented across multiple packets (e.g., MIME multipart data).
- **Process:**
  1. Export the data section from each relevant packet as individual files (e.g., `file0`, `file1`).
  2. Reassemble them using a command line tool.

- **Reassembly Example:** `cat file* > complete_file` (Concatenates all parts back into the original file).

## FTP

- When files are transferred via FTP, look for **FTP-DATA** packets. Following this stream or extracting the object will recover the transferred file.

- 
1. Open the PCAP file located in the `labfiles` directory and analyze the traffic in this file.
  2. What is the stream index number of the first FTP `PASS` command?

Answer: 190

3. What source port was used to transfer the FTP text file?

Answer: 49233

4. Open the **export HTTP objects** window.
5. What is the size of the first image/gif (in bytes) in the HTTP objects list?

Answer: 43

6. Export the file `show_ads.js`. Once saved, run `md5sum` on this file.
7. What are the last five characters of this hash?

Answer: c6414

8. Open the export SMB objects window and export the `.docx` file. Once saved, use `unzip` to extract the contents of the file.
9. Inside one of the XML files is a flag. What is the flag?

Answer: 79e0f325804dafbdaef73b3b17c0fd8d

---

## TLS Traffic

---

### Purpose

- **Why:** Modern network traffic (especially HTTPS) is often encrypted with TLS, making packet contents unreadable.

- **Goal:** To decrypt TLS traffic in Wireshark using session keys to analyze hidden malicious activity.

## TLS Overview

- **TLS (Transport Layer Security):** A cryptographic protocol that encrypts communication between two hosts (successor to SSL).
- **Dual Purpose:**
  1. **Encryption:** Protects data in transit.
  2. **Identity Assurance:** Provides trust via certificates.
- **Attacker Use:** Cybercriminals also use TLS to hide malicious traffic and make phishing sites appear "secure."

## Decrypting TLS in Wireshark

Wireshark can decrypt TLS if provided with the appropriate secrets. Two main methods:

1. **Key Log File (Pre-Master Secret) - Focus of this lab:** Uses per-session keys logged by the browser or client.
2. **RSA Private Key:** Uses the server's private key (less common).

## Method: Using a Key Log File

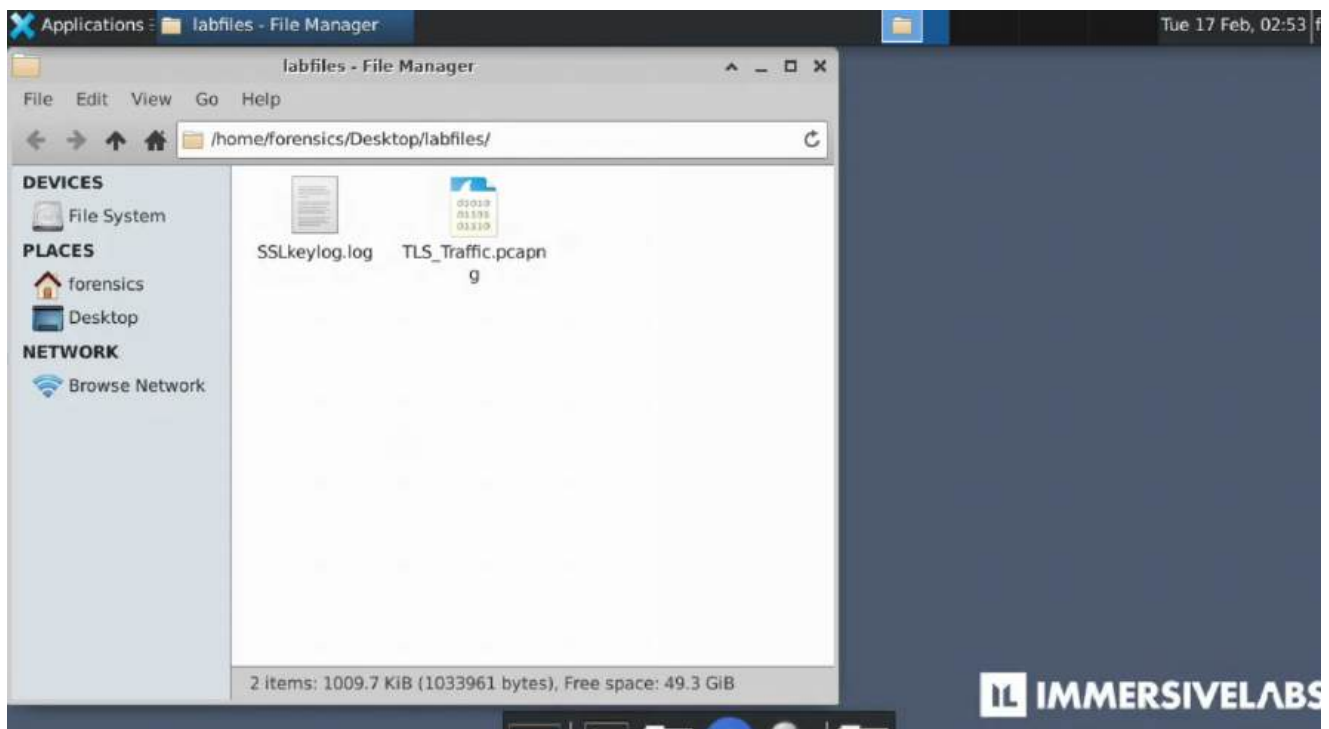
- **What it is:** A text file containing session secrets (keys) that allow Wireshark to decrypt the traffic.
- **Source:** Can be generated by setting an environment variable in the browser before capture, or extracted post-capture using third-party tools.
- **How to configure in Wireshark:**
  1. Go to **Edit** → **Preferences**.
  2. Navigate to **Protocols** → **SSL** (or **TLS** if available).
  3. In the "(Pre)-Master-Secret log filename" field, browse and select the key log file.

- 
1. What does TLS stand for?

Answer: Transport Layer Security

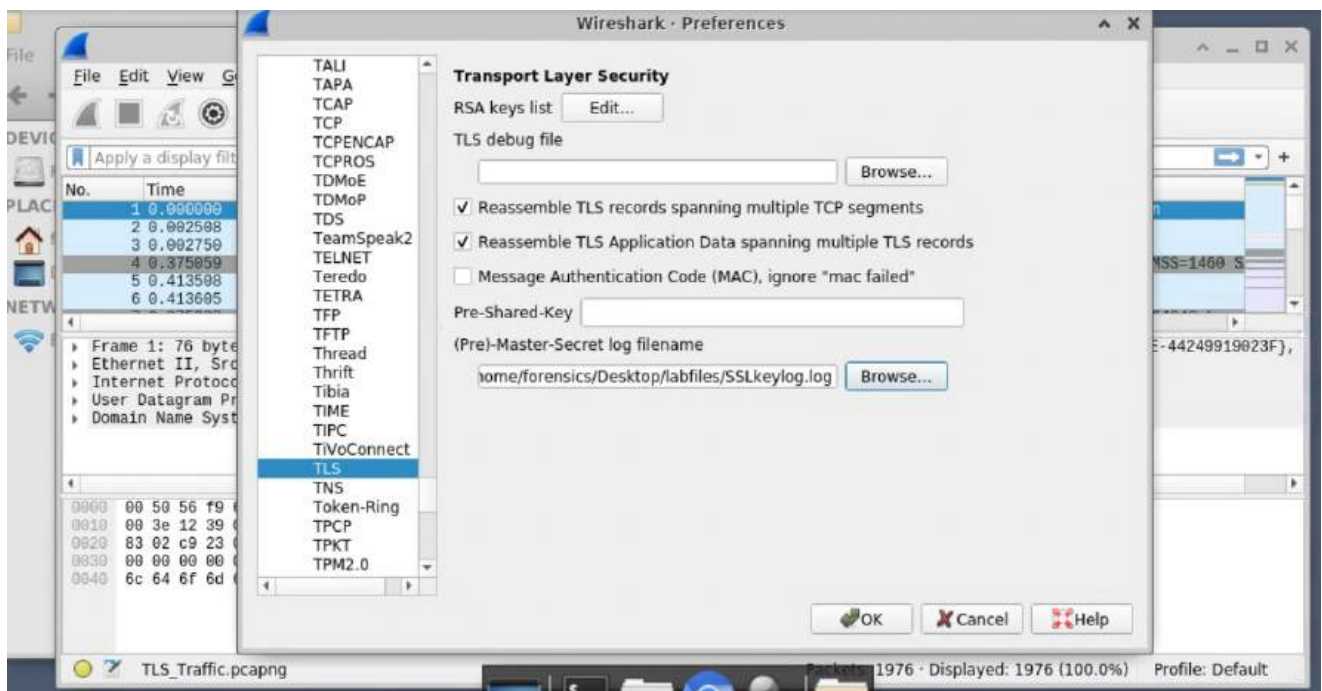
---

2. Open the pcap file located in the `labfiles` directory.



### 3. Decrypt SSL traffic in the Wireshark application.

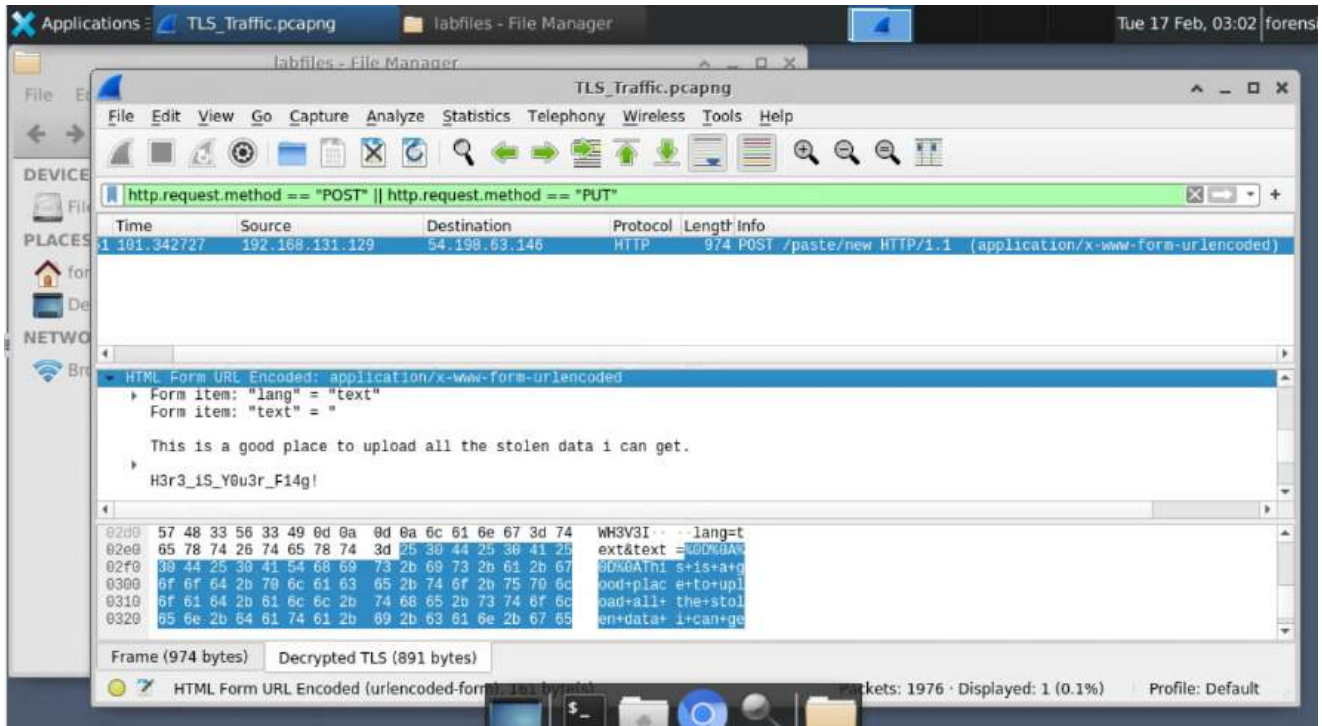
- **Open Wireshark** and load your lab PCAP file.
- Go to **Edit > Preferences** (on a Mac: **Wireshark > Settings**).
- On the left-hand menu, expand **Protocols**.
- Scroll down to **TLS** (in older versions of Wireshark, this is labeled **SSL**).
- Find the field labeled **(Pre)-Master Secret log filename**.
- Click **Browse...** and select your `sslkeylog.log` file.
- Click **OK**.



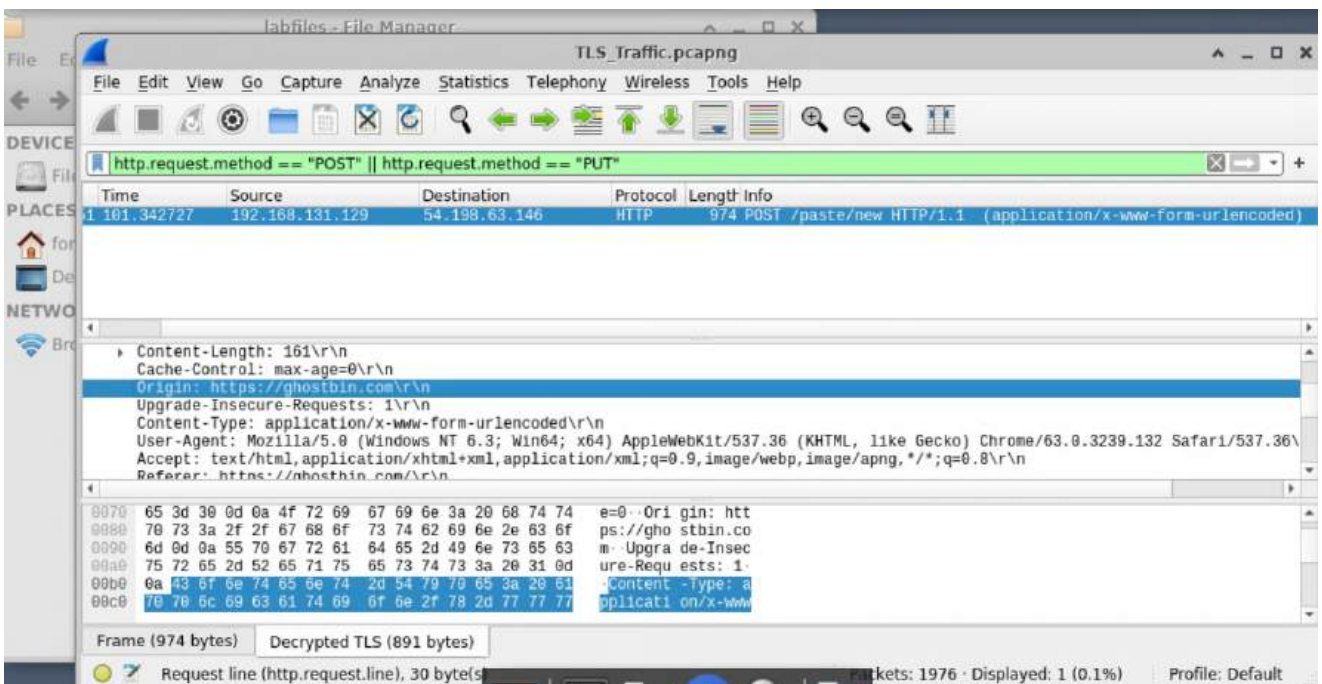


4. Identify the online service that was used to exfiltrate stolen data.

`http.request.method == "POST" || http.request.method == "PUT"`



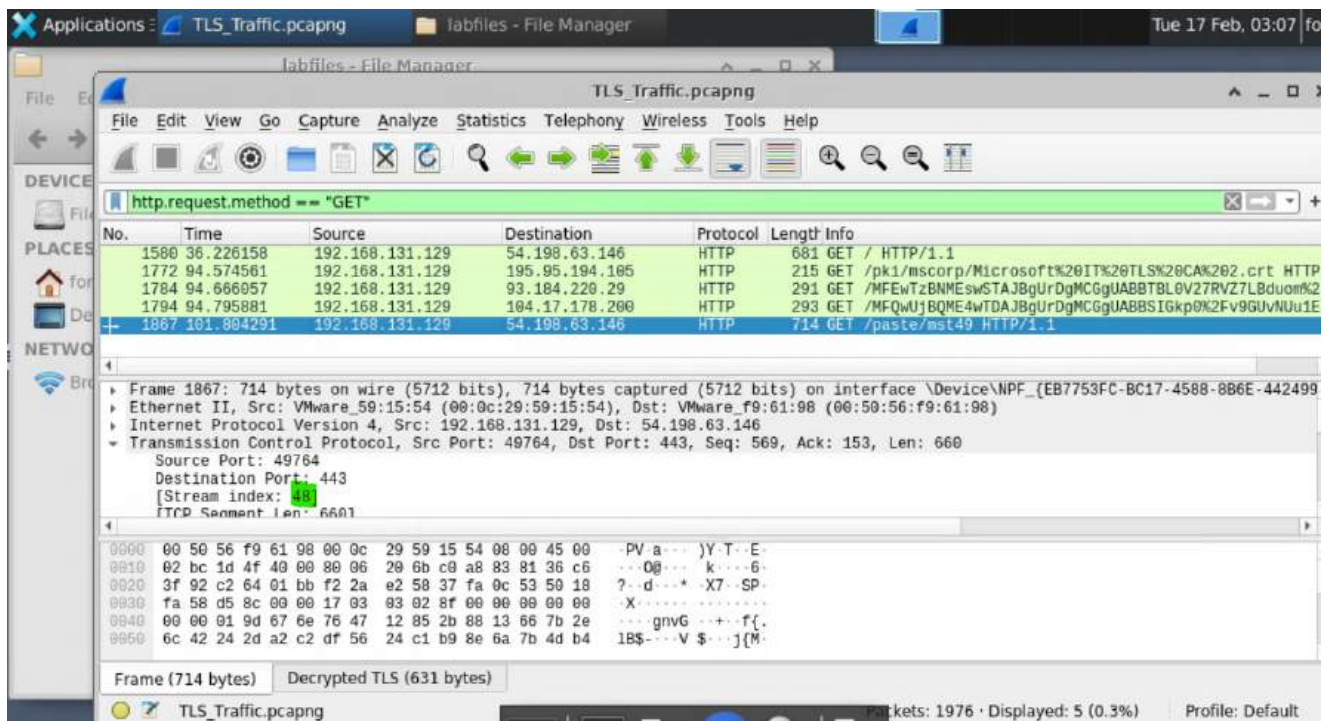
5. What domain was used to exfiltrate the data?



Answer: ghostbin.com

6. What is the TCP stream index number of the GET request in which the stolen data was submitted?

`http.request.method == "GET"`



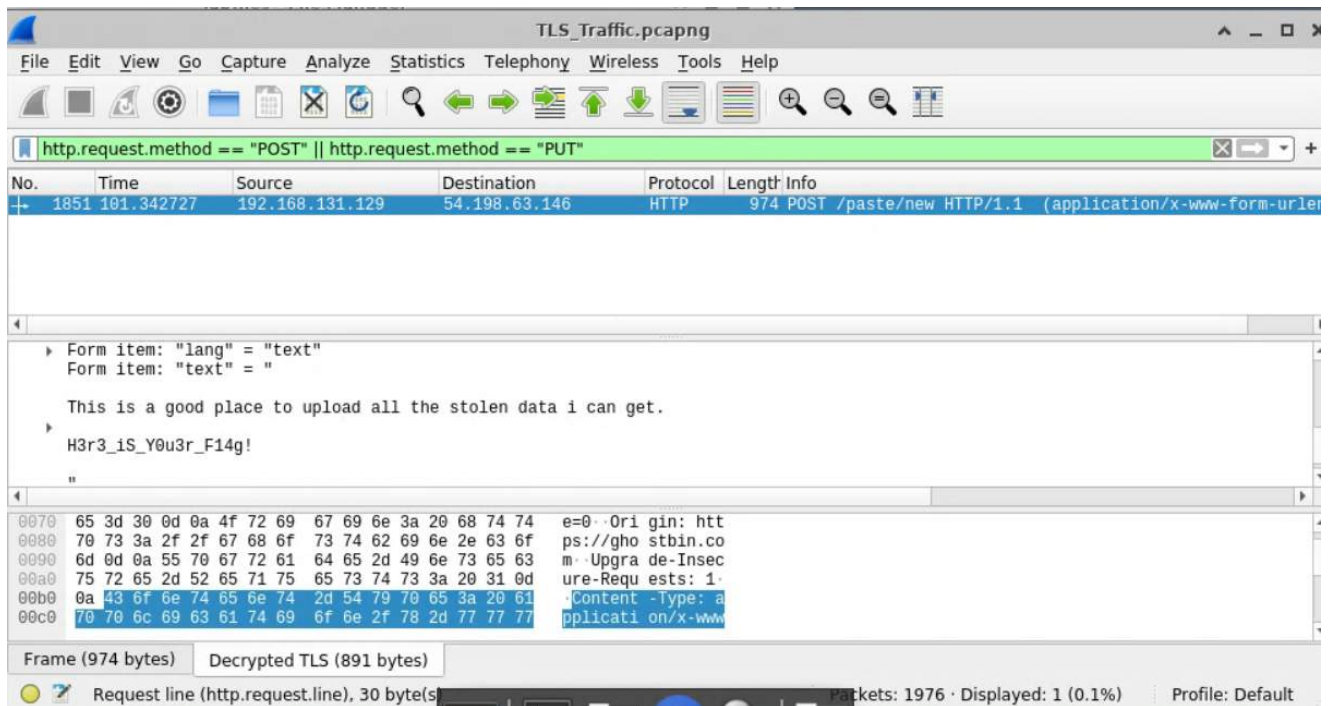
Answer: 48

7. What is the filename of the submitted html file?

Answer: mst49

8. Identify the flag in the POSTed data.

`http.request.method == "POST" || http.request.method == "PUT"`



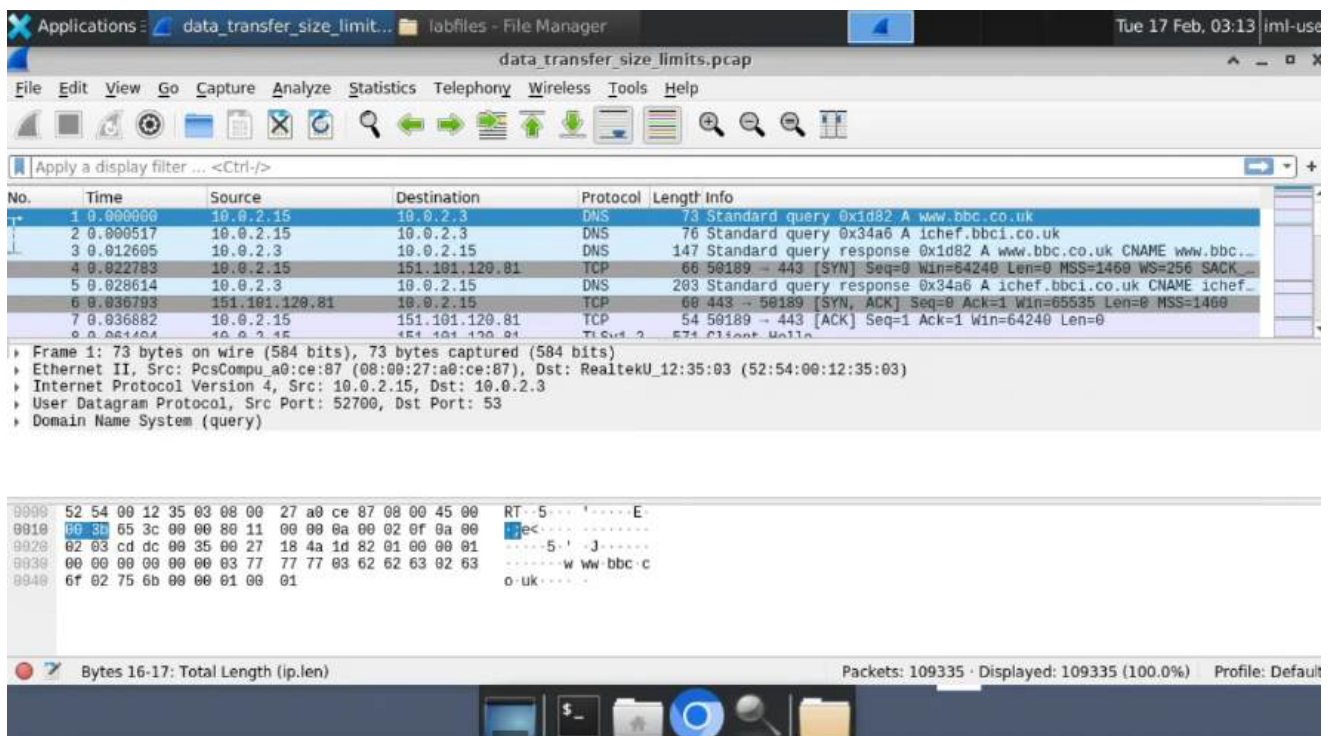
9. What is the hidden flag posted in place of the stolen data?

Answer: H3r3\_iS\_Y0u3r\_F14g!

## Demonstrate Your Skills

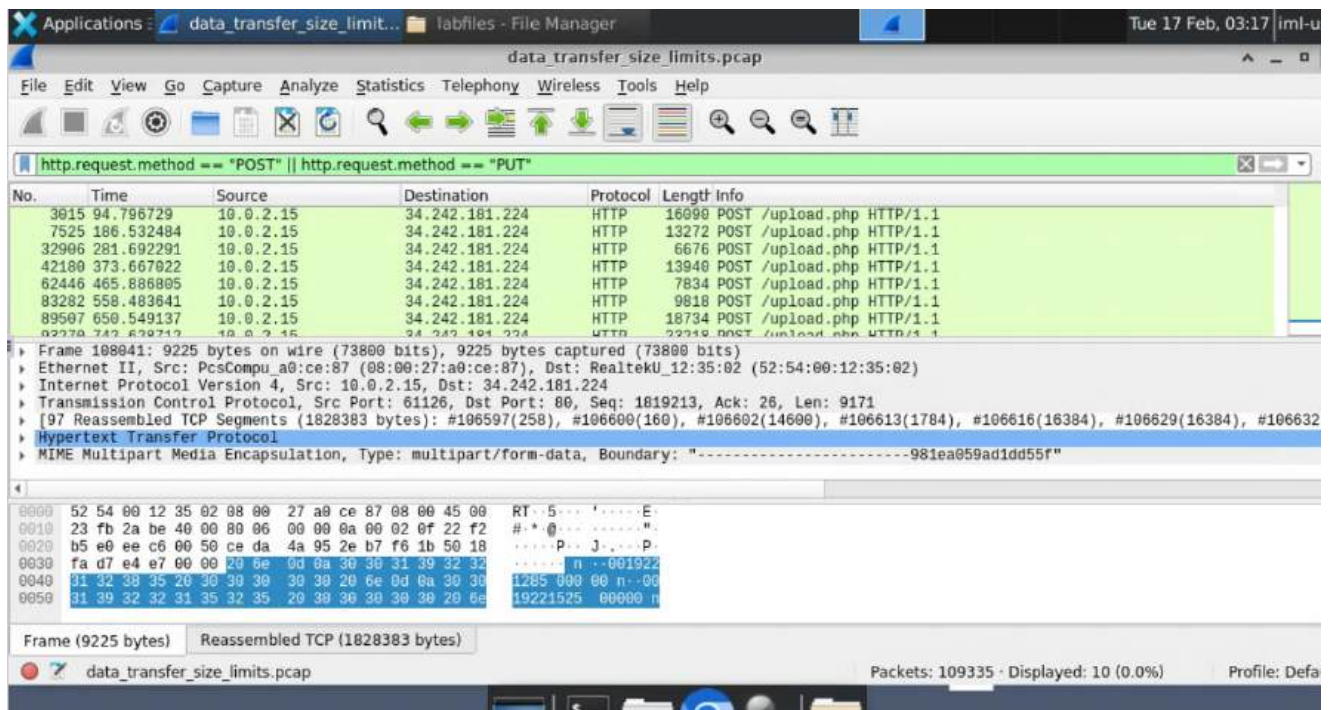
1. Analyze the PCAP file located in the `labfiles` directory.





2. What is the IP address of the server the attacker was attempting to exfiltrate information to?

http.request.method == "POST" || http.request.method == "PUT"



Answer: 34.242.181.224

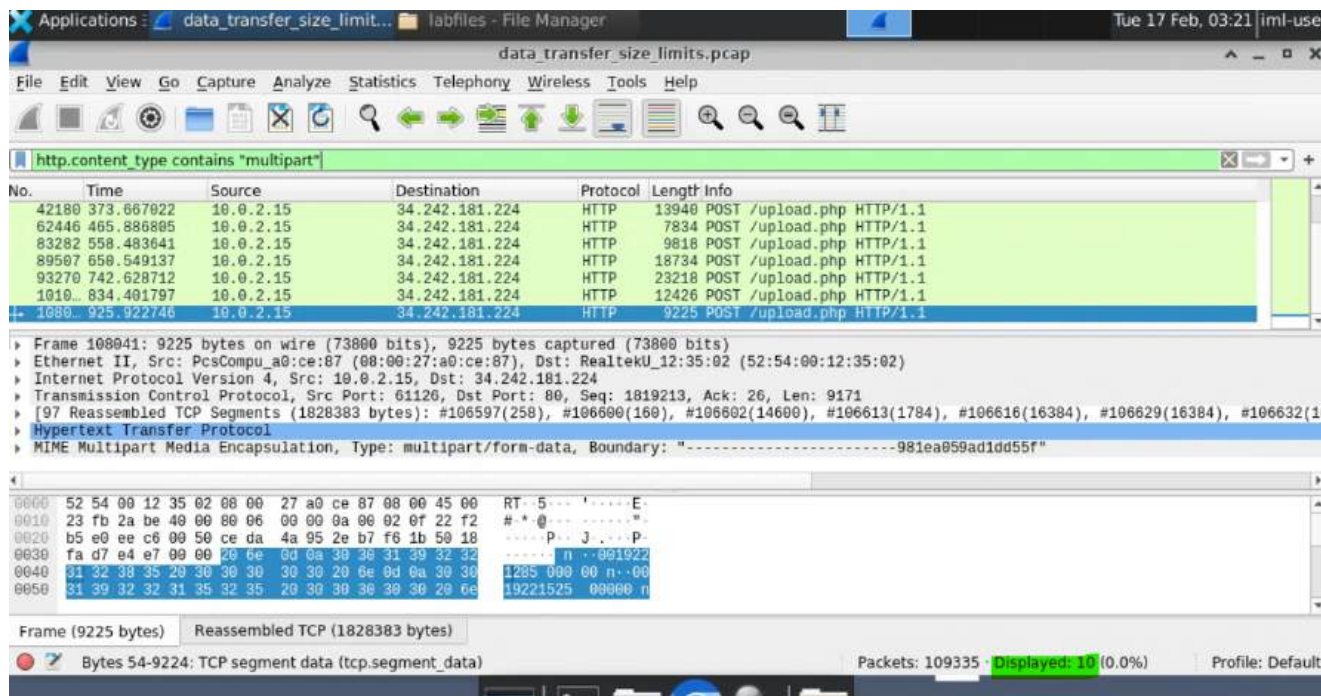
3. Identify the network packets where the attacker was attempting to steal information.



4. Identify how many files the attacker was attempting to exfiltrate.

5. How many files did the attacker attempt to send to the remote server?

http.content\_type contains "multipart"



Answer: 10

6. Based on the time between each file transfer, how many seconds did the attacker wait before attempting to send each file to the remote server?

Calculate the time by subtracting the time stamps of the packets.

They are generally between 90-95 seconds.

Example:

```
Frame 89507: 18734 bytes on wire (149872 bits), 18734 bytes captured (149872 bi
Encapsulation type: Ethernet (1)
Arrival Time: Mar 8, 2019 10:20:00.965535000 UTC
[Time shift for this packet: 0.000000000 seconds]
Epoch Time: 1552040400.965535000 seconds
[Time delta from previous captured frame: 0.000064000 seconds]
[Time delta from previous displayed frame: 92.065496000 seconds]
[Time since reference or first frame: 650.549137000 seconds]
```

```

▼ Frame 93270: 23218 bytes on wire (185744 bits), 23218 bytes captured (185744 bits)
 Encapsulation type: Ethernet (1)
 Arrival Time: Mar 8, 2019 10:21:33.045110000 UTC
 [Time shift for this packet: 0.000000000 seconds]
 Epoch Time: 1552040493.045110000 seconds
 [Time delta from previous captured frame: 0.000155000 seconds]
 [Time delta from previous displayed frame: 92.079575000 seconds]
 [Time since reference or first frame: 742.628712000 seconds]

▼ Frame 101096: 12426 bytes on wire (99408 bits), 12426 bytes captured (99408 bits)
 Encapsulation type: Ethernet (1)
 Arrival Time: Mar 8, 2019 10:23:04.818195000 UTC
 [Time shift for this packet: 0.000000000 seconds]
 Epoch Time: 1552040584.818195000 seconds
 [Time delta from previous captured frame: 0.000089000 seconds]
 [Time delta from previous displayed frame: 91.773085000 seconds]
 [Time since reference or first frame: 834.401797000 seconds]

```

Answer: 90

7. In bytes, what size limit did the attacker apply to the files they attempted to exfiltrate?

The screenshot shows a Wireshark packet capture of an HTTP POST request to /upload.php. The packet list shows a POST request with a length of 18734 bytes. The packet details show a multipart/form-data request with a file named 'file.007'. The data field shows the file content, which is a 2097152 byte file.

| No.    | Time       | Source    | Destination    | Protocol | Length | Info                      |
|--------|------------|-----------|----------------|----------|--------|---------------------------|
| 42186  | 373.667022 | 10.0.2.15 | 34.242.181.224 | HTTP     | 13940  | POST /upload.php HTTP/1.1 |
| 62446  | 465.886895 | 10.0.2.15 | 34.242.181.224 | HTTP     | 7834   | POST /upload.php HTTP/1.1 |
| 83282  | 558.483641 | 10.0.2.15 | 34.242.181.224 | HTTP     | 9818   | POST /upload.php HTTP/1.1 |
| 89507  | 659.549137 | 10.0.2.15 | 34.242.181.224 | HTTP     | 18734  | POST /upload.php HTTP/1.1 |
| 93270  | 742.628712 | 10.0.2.15 | 34.242.181.224 | HTTP     | 23218  | POST /upload.php HTTP/1.1 |
| 101096 | 834.401797 | 10.0.2.15 | 34.242.181.224 | HTTP     | 12426  | POST /upload.php HTTP/1.1 |
| 10880  | 925.922746 | 10.0.2.15 | 34.242.181.224 | HTTP     | 9225   | POST /upload.php HTTP/1.1 |

Packet 89507 details:

```

[Type: multipart/form-data]
First boundary: -----08711d037b4a4caa\r\n
 Encapsulated multipart part: (application/octet-stream)
 Content-Disposition: form-data; name="fileToUpload"; filename="file.007"\r\n
 Content-Type: application/octet-stream\r\n\r\n
 Data (2097152 bytes)
 Last boundary: \r\n-----08711d037b4a4caa--\r\n

```

Packet 89507 data:

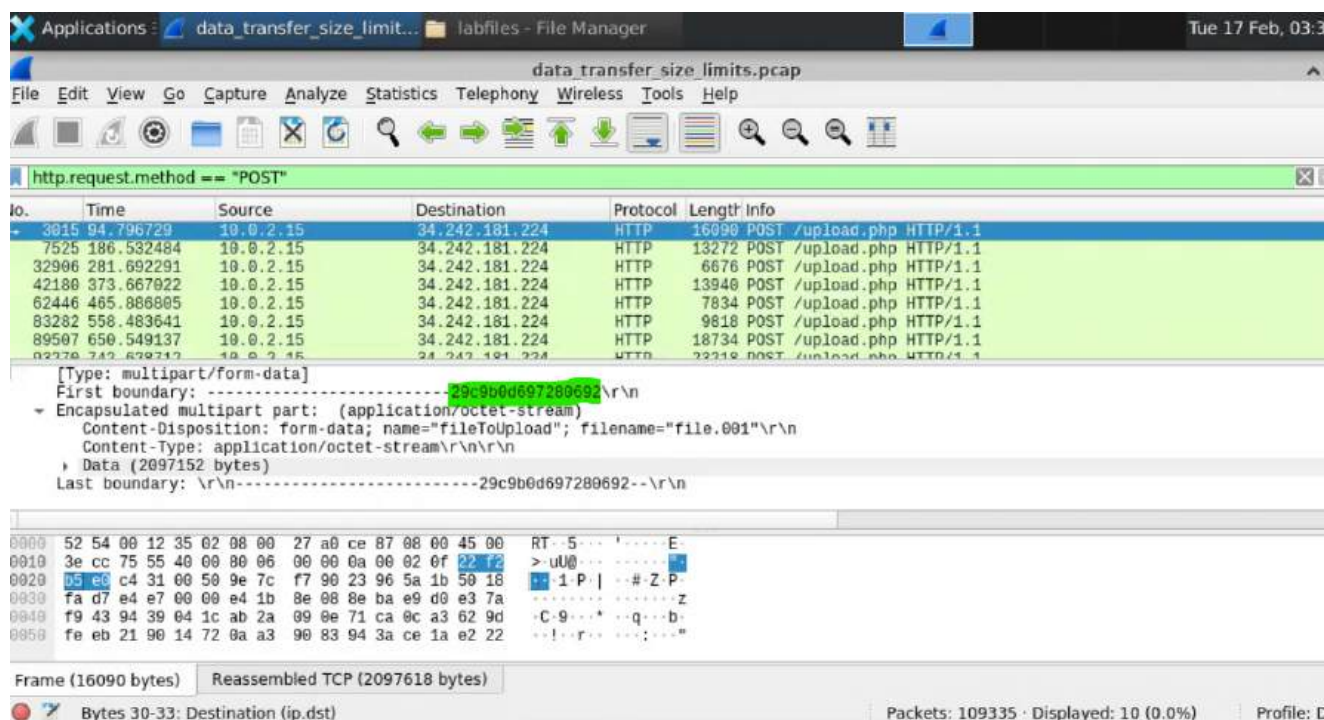
```

52 54 00 12 35 02 08 00 27 a0 ce 87 08 00 45 00 RT-5...E
49 20 19 70 40 00 08 06 00 00 0a 00 02 0f 22 f2 I pg...
b5 e0 ee 6f 00 50 99 90 ff ab 2c 73 e2 1b 50 18 o P...s P
fa d7 e4 e7 00 00 38 1b a7 69 b9 e3 7b 88 7a a6 ...1...Z
f7 dc 89 db a1 fc 37 41 fb a1 c6 90 9e c7 f7 9a ...1...7A
4f dd 9c 22 3a 69 ec 11 17 91 f3 b2 e2 52 ea 17 0...1...R

```

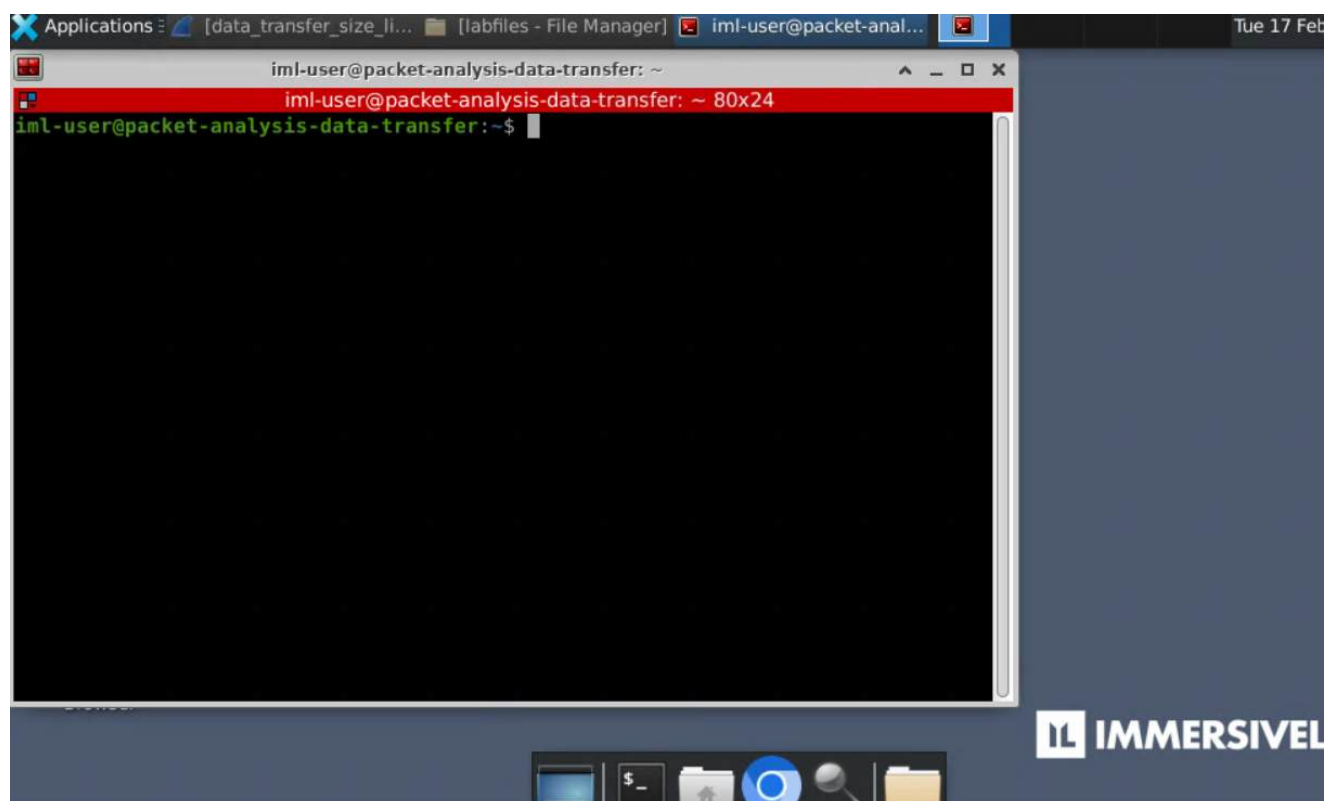
Answer: 2097152

8. What is the boundary for the data in the first HTTP packet sent to the remote server?



Answer: 29c9b0d697280692

- Using the Linux command line or another appropriate tool available on the desktop, rebuild the original file and identify what information the attacker was attempting to steal.



10. Rebuild the file using the segments found in the PCAP. What is the MD5 hash of this file?

File > Export Packet Bytes.. > Save all the packet bytes

```
cat 1 2 3 4 5 6 7 8 9 10 > rebuilt_file
```

```
md5sum rebuilt_file
```

Answer: 8cbf90aeab2c93b2819fcfd6262b2cdb

---

11. What file type did the attacker attempt to exfiltrate?

Answer: pdf

---

12. What is the title of the document targeted in this attack?

Answer: **Final Flight Plan**

---